**IF2211** STRATEGI ALGORITMA

# LAPORAN TUGAS KECIL 2

**Disusun oleh:**

Hakam Avicena Mustain (13524075)

Rava Khoman Tuah Saragih (13524107)


**Dikumpulkan pada:**

28 Maret 2026


**PROGRAM STUDI TEKNIK INFORMATIKA**

**SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA**

**INSTITUT TEKNOLOGI BANDUNG**

**2026**

# Daftar Isi

**Daftar Isi** 3
**BAB I** 1
**LANDASAN TEORI** 1
1.1. Algoritma Divide and Conquer 1
1.2. Axis-Aligned Bounding Box (AABB) 2
1.3. Separating Axis Theorem (SAT) 3
1.4. Konsep Proyeksi dan Overlap 4
1.5. Pemrograman Konkuren pada C++ 5
1.5.1. std::thread 5
1.5.2. std::mutex 5
**BAB II** 5
**METODOLOGI** 5
2.1. Langkah 1: Translasi ke Pusat AABB 6
2.2. Langkah 2: Perhitungan Edge Segitiga 6
2.3. Langkah 3: Pengujian 13 Sumbu Kandidat 7
a. Tiga Sumbu Koordinat AABB 7
b. Satu Sumbu Normal Segitiga 7
c. Sembilan Sumbu Cross Product 7
2.4. Langkah 4: Penentuan Hasil Intersection 8
**BAB III** 9
**PERANCANGAN DAN IMPLEMENTASI** 9
3.1. Langkah-langkah Program 9
3.2. Pseudocode Algoritma 10
3.2.1. Algoritma Pembangunan Octree (buildOctree) 10
3.2.2. Algoritma Pengujian Perpotongan SAT (triangleIntersectsAABB) 12
3.2.3. Algoritma Generate Geometri Voxel (generateVoxelMesh) 13
3.3. Implementasi Konkuren 13
3.3.1. Strategi Konkuren pada buildOctree 13
3.3.2. Strategi Konkuren pada generateVoxelMesh 14
3.3.3. Implementasi OBJ Viewer Interaktif 15
**3.3.3 Source Code dan Pseudocode** 16
3.3.3.1 Source Code 16
3.3.3.2 Pseudocode 49
**BAB III** 78
**PENGUJIAN** 78
2.1. Pengujian Algoritma Divide and Conquer 78
2.1.1 Kasus Uji 1 78
2.1.2 Kasus Uji 2 79

2.1.3 Kasus Uji 3 ..... 80
2.1.4 Kasus Uji 4 ..... 82
2.1.5 Kasus Uji 5 ..... 83
2.2. Analisis Hasil Pengujian ..... 84
**BAB IV ..... 85**
3.1. Kesimpulan ..... 85
**Lampiran ..... 87**
Pernyataan tidak melakukan kecurangan yang ditandatangani ..... 87
Tautan repository GitHub dan video ..... 87
Tabel Penyelesaian spesifikasi wajib dan bonus ..... 87
**Daftar Pustaka ..... 89**

# BAB I

# LANDASAN TEORI

## 1.1. Algoritma Divide and Conquer

Divide and Conquer adalah paradigma desain algoritma yang bekerja dengan cara memecah suatu masalah besar menjadi beberapa sub-masalah yang lebih kecil, menyelesaikan sub-masalah secara mandiri (atau rekursif), lalu menggabungkan hasil dari setiap sub-masalah menjadi solusi akhir. Algoritma ini secara umum terdiri dari tiga tahap utama:

1. Divide: Memecah masalah menjadi sub-masalah yang lebih kecil.
2. Conquer: Menyelesaikan setiap sub-masalah secara rekursif. Apabila sub-masalah sudah cukup kecil, selesaikan secara langsung (base case).
3. Combine: Menggabungkan solusi sub-masalah menjadi solusi untuk masalah yang lebih besar.

Dalam konteks pembangunan Octree untuk voxelisasi, ketiga tahap tersebut diterapkan sebagai berikut:

1. Divide: Membagi ruang kubus (bounding box) yang melingkupi objek 3D menjadi 8 sub-kubus yang lebih kecil dan sama besar (oktan), dengan cara membelah ruang di titik tengah pada sumbu X, Y, dan Z secara serentak.
2. Conquer: Pada setiap oktan, dilakukan pengujian perpotongan (intersection) antara segitiga-segitiga penyusun mesh objek dengan batas ruang oktan tersebut menggunakan Separating Axis Theorem (SAT). Proses ini dilakukan secara rekursif hingga mencapai kedalaman maksimum yang telah ditentukan, atau hingga oktan tidak lagi mengandung segitiga apapun (pruning).

3. Combine: Mengumpulkan seluruh leaf node di kedalaman maksimum yang masih memiliki segitiga sebagai representasi voxel. Kumpulan voxel ini kemudian di-generate menjadi geometri kubus dan disatukan menjadi file .obj output.

Penggunaan Divide and Conquer pada masalah ini memberikan efisiensi yang signifikan. Alih-alih menguji setiap segitiga terhadap setiap kemungkinan posisi voxel secara brute force, Octree memungkinkan pemangkasan (pruning) seluruh cabang pohon yang jelas-jelas tidak berpotongan dengan mesh, sehingga mengurangi jumlah komputasi secara drastis, terutama pada kedalaman pohon yang besar.

### 1.2. Axis-Aligned Bounding Box (AABB)

Axis-Aligned Bounding Box (AABB) adalah representasi geometri berupa kotak pembatas tiga dimensi yang sisi-sisinya selalu sejajar dengan sumbu koordinat utama, yaitu sumbu X, Y, dan Z. Kata "Axis-Aligned" mengacu pada sifat ini, yaitu kotak tidak pernah dirotasikan terhadap sistem koordinat.

Karena tidak ada rotasi, sebuah AABB cukup direpresentasikan oleh dua titik sudut yang berlawanan, yaitu titik minimum dan titik maksimum. Titik minimum menyimpan koordinat terkecil pada ketiga sumbu (x_min, y_min, z_min), sedangkan titik maksimum menyimpan koordinat terbesar (x_max, y_max, z_max). Dalam kode program, ini direpresentasikan sebagai struct AABB dengan dua field bertipe Vec3.

Seluruh node dalam struktur Octree yang dibangun pada program ini berbentuk AABB, dari node akar (root) yang melingkupi seluruh objek 3D, hingga node daun (leaf) di kedalaman maksimum yang menjadi voxel akhir. Karena AABB selalu sejajar dengan sumbu koordinat, arah normal keenam sisi-nya selalu tetap, yaitu (1,0,0) untuk arah X positif, (-1,0,0) untuk X negatif, dan seterusnya. Sifat ini dimanfaatkan langsung dalam pengujian perpotongan menggunakan Separating Axis Theorem.

Untuk memastikan seluruh objek 3D masuk ke dalam satu node akar, terlebih dahulu dihitung AABB terkecil yang melingkupi semua vertex objek (computeAABB), kemudian AABB tersebut diperluas menjadi kubus sempurna dengan sisi yang sama panjang pada ketiga sumbu (expandToCube). Langkah perluasan ini penting karena pembagian Octree mengharuskan setiap node dibagi menjadi 8 oktan yang identik, yang hanya mungkin dilakukan pada kubus sempurna.

### 1.3. Separating Axis Theorem (SAT)

Separating Axis Theorem (SAT) adalah teorema geometri yang menyatakan bahwa dua objek konveks tidak saling berpotongan (intersect) jika dan hanya jika terdapat setidaknya satu sumbu yang dapat memisahkan keduanya. Sumbu pemisah tersebut didefinisikan sebagai sumbu di mana proyeksi kedua objek ke sumbu itu tidak beririsan (terdapat celah di antara keduanya).

Intuisi dari teorema ini sederhana: apabila dua benda tidak bersentuhan, pasti ada ruang kosong di antara keduanya. Ruang kosong tersebut selalu dapat digambarkan sebagai bidang pemisah, dan vektor normal dari bidang tersebut adalah sumbu pemisahnya. Sebaliknya, apabila tidak ditemukan satupun sumbu yang mampu memisahkan kedua objek dari semua arah yang diuji, maka kedua objek pasti berpotongan.

Untuk menguji perpotongan antara sebuah segitiga dan sebuah AABB, diperlukan pengujian pada 13 sumbu kandidat yang terdiri dari:

1. 3 sumbu koordinat AABB: arah sumbu X (1,0,0), Y (0,1,0), dan Z (0,0,1). Pengujian pada ketiga sumbu ini secara efektif memeriksa apakah segitiga berada di luar rentang AABB pada masing-masing arah koordinat.
2. 1 sumbu normal segitiga: vektor tegak lurus terhadap bidang segitiga, diperoleh dari perkalian silang dua vektor sisi segitiga. Sumbu ini menguji apakah AABB menembus bidang yang memuat segitiga.
3. 9 sumbu dari perkalian silang antara tiga vektor sisi segitiga (e0, e1, e2) dengan tiga sumbu AABB (X, Y, Z): menghasilkan 3×3 = 9 sumbu. Sumbu-sumbu ini

diperlukan untuk mendeteksi kasus geometri ekstrem di mana keempat sumbu sebelumnya tidak mencukupi, khususnya ketika segitiga berorientasi miring terhadap AABB.

Apabila pada salah satu dari 13 sumbu ditemukan celah antara proyeksi segitiga dan proyeksi AABB, maka kedua objek dipastikan tidak berpotongan dan algoritma dapat dihentikan lebih awal (early exit). Apabila seluruh 13 sumbu menunjukkan tumpang tindih proyeksi, maka segitiga dan AABB dinyatakan berpotongan.

### 1.4. Konsep Proyeksi dan Overlap

Proyeksi dalam konteks SAT adalah proses memetakan sebuah objek tiga dimensi ke dalam satu dimensi sepanjang suatu sumbu tertentu. Hasil proyeksi berupa sebuah interval, yaitu nilai minimum dan maksimum dari seluruh titik objek ketika diproyeksikan ke arah sumbu yang dimaksud.

Proyeksi sebuah titik v terhadap sumbu a dihitung menggunakan perkalian dot product: $p = v \cdot a$. Untuk segitiga dengan tiga titik sudut, diperoleh tiga nilai proyeksi yang membentuk interval $[\min(p0, p1, p2), \max(p0, p1, p2)]$.

Proyeksi AABB yang telah ditranslasikan ke titik asal terhadap suatu sumbu a direpresentasikan sebagai interval simetris $[-r, +r]$, di mana nilai r dihitung sebagai:

$$r = |half.x \times a.x| + |half.y \times a.y| + |half.z \times a.z|$$

di mana half adalah setengah ukuran AABB pada masing-masing sumbu (extents), dan nilai r merepresentasikan sejauh mana AABB "memanjang" dari pusat ke arah sumbu a. Dua interval dikatakan tidak beririsan (terdapat celah pemisah) apabila nilai minimum proyeksi segitiga lebih besar dari r, atau nilai maksimum proyeksi segitiga lebih kecil dari −r. Kondisi batas (menyentuh tepat di r atau −r) dianggap berpotongan.

## 1.5. Pemrograman Konkuren pada C++

Pemrograman konkuren memungkinkan program untuk menjalankan beberapa tugas secara paralel pada CPU multi-core, sehingga dapat mempercepat proses komputasi yang bersifat independen satu sama lain. Pada program ini, digunakan fitur konkuren bawaan C++ dari header <thread> dan <mutex> tanpa library eksternal.

### 1.5.1. std::thread

std::thread adalah representasi dari sebuah thread eksekusi dalam C++ standar. Sebuah thread dapat dibuat dengan memberikan fungsi (atau lambda) beserta argumennya ke konstruktor std::thread. Setelah dibuat, thread berjalan secara independen secara paralel dengan thread lain. Fungsi join() digunakan untuk menunggu sebuah thread selesai sebelum melanjutkan eksekusi di thread pemanggil, sehingga memastikan seluruh hasil komputasi sudah tersedia sebelum digabungkan.

### 1.5.2. std::mutex

std::mutex (mutual exclusion) adalah mekanisme sinkronisasi yang digunakan untuk mengatur akses ke sumber daya yang digunakan bersama oleh beberapa thread (shared memory). Mutex memastikan bahwa hanya satu thread yang dapat mengakses atau memodifikasi data kritis pada satu waktu, sehingga menghindari race condition, yaitu kondisi di mana hasil akhir bergantung pada urutan eksekusi thread yang tidak dapat diprediksi. Pada program ini, penggunaan mutex diminimalisasi dengan strategi per-thread buffer sehingga tidak menjadi bottleneck.

# BAB II

# METODOLOGI

Algoritma pengujian perpotongan antara segitiga dan AABB yang digunakan dalam program ini mengikuti metode yang dikembangkan oleh Tomas Akenine-Möller, yang secara matematis menguji 13 sumbu kandidat menggunakan prinsip SAT. Berikut adalah langkah-langkahnya secara rinci.

### 2.1. Langkah 1: Translasi ke Pusat AABB

Langkah pertama adalah menggeser seluruh koordinat ke sistem referensi baru dengan pusat AABB berada di titik asal (0, 0, 0). Tujuan dari langkah ini adalah menyederhanakan perhitungan proyeksi AABB, karena setelah translasi, interval proyeksi AABB pada semua sumbu selalu simetris terhadap nol, yaitu [−r, +r].

Pusat AABB dihitung sebagai rata-rata dari titik minimum dan maksimum, kemudian setengah ukuran kotak (extents) dihitung sebagai setengah selisihnya. Ketiga titik sudut segitiga kemudian digeser dengan mengurangkan pusat AABB:

```
center = (box.min + box.max) × 0.5
extents = (box.max − box.min) × 0.5
v0 = v0 − center,    v1 = v1 − center,    v2 = v2 − center
```

Setelah translasi ini, seluruh perhitungan proyeksi AABB dapat dilakukan secara seragam menggunakan satu nilai r untuk setiap sumbu, tanpa perlu memperhitungkan posisi absolut AABB di ruang 3D.

### 2.2. Langkah 2: Perhitungan Edge Segitiga

Setelah translasi, dihitung tiga vektor sisi (edge) segitiga yang akan digunakan untuk membentuk sumbu-sumbu kandidat SAT. Ketiga edge didefinisikan sebagai selisih antara pasangan titik segitiga yang telah ditranslasi:

```
e0 = v1 − v0,    e1 = v2 − v1,    e2 = v0 − v2
```

Vektor e0 dan e1 bersama-sama berada pada bidang yang sama dan digunakan untuk menghitung normal bidang segitiga melalui perkalian silang (cross product).

Ketiga vektor edge ini juga akan di-cross product dengan ketiga sumbu koordinat kotak untuk menghasilkan 9 sumbu kandidat tambahan.

## 2.3. Langkah 3: Pengujian 13 Sumbu Kandidat

### a. Tiga Sumbu Koordinat AABB

Tiga sumbu pertama adalah arah sumbu koordinat utama: X = (1,0,0), Y = (0,1,0), Z = (0,0,1). Karena AABB selalu sejajar dengan sumbu koordinat, pengujian pada ketiga sumbu ini secara langsung memeriksa apakah segitiga berada di luar rentang AABB pada masing-masing arah. Implementasinya adalah membandingkan nilai maksimum dan minimum dari proyeksi ketiga vertex segitiga terhadap sumbu tersebut dengan nilai extents pada sumbu yang bersangkutan. Jika rentang proyeksi segitiga sepenuhnya berada di luar rentang extents AABB, maka terdapat sumbu pemisah.

### b. Satu Sumbu Normal Segitiga

Sumbu keempat adalah normal bidang segitiga, diperoleh dari perkalian silang antara e0 dan e1. Sumbu ini menguji apakah AABB memiliki jangkauan yang cukup pada arah tegak lurus bidang segitiga untuk menyentuh bidang tersebut. Nilai r untuk sumbu ini dihitung sebagai:

```
r = extents.x × |normal.x| + extents.y × |normal.y| + extents.z ×
                                |normal.z|
```

Kemudian d = dot(normal, v0) adalah jarak bidang segitiga dari titik asal. Jika d > r atau d < −r, terdapat sumbu pemisah.

### c. Sembilan Sumbu Cross Product

Sembilan sumbu berikutnya diperoleh dari perkalian silang antara masing-masing tiga sisi segitiga (e0, e1, e2) dengan masing-masing tiga sumbu koordinat AABB (X, Y, Z), menghasilkan 3 × 3 = 9 kombinasi. Sumbu-sumbu ini diperlukan untuk mendeteksi kasus geometri ekstrem, khususnya ketika segitiga berorientasi

diagonal terhadap AABB sehingga keempat sumbu sebelumnya tidak mencukupi untuk menentukan ada atau tidaknya celah pemisah.

Untuk setiap sumbu hasil cross product, dilakukan proyeksi segitiga ke sumbu tersebut dan dihitung nilai r-nya. Karena salah satu komponen sumbu cross product dari "edge × sumbu-koordinat" selalu nol, perhitungan dapat disederhanakan secara aljabar sehingga hanya dua dari tiga vertex segitiga yang perlu diproyeksikan (bukan tiga), menghemat sejumlah operasi floating point.

### 2.4. Langkah 4: Penentuan Hasil Intersection

Setelah seluruh 13 sumbu diuji, penentuan hasil perpotongan mengikuti prinsip SAT secara ketat. Algoritma ini menggunakan strategi early exit: begitu satu celah (gap) ditemukan antara interval proyeksi segitiga dan interval proyeksi AABB pada salah satu sumbu, fungsi langsung mengembalikan false tanpa perlu menguji sumbu-sumbu berikutnya. Hal ini menjadikan algoritma sangat efisien dalam kasus di mana segitiga dan AABB jelas-jelas tidak berpotongan.

Apabila seluruh 13 sumbu telah diuji dan tidak ditemukan celah apapun, maka segitiga dan AABB dinyatakan berpotongan dan fungsi mengembalikan true. Kondisi batas di mana nilai proyeksi tepat sama dengan r juga dianggap berpotongan, karena segitiga menyentuh tepat pada permukaan AABB.

Dalam konteks pembangunan Octree, hasil pengujian ini menentukan apakah sebuah node perlu ditelusuri lebih lanjut. Node yang tidak berpotongan dengan segitiga manapun dapat dipangkas (pruning) seluruh sub-pohonnya, sehingga secara signifikan mengurangi jumlah komputasi yang diperlukan, terutama pada kedalaman pohon yang besar.

# BAB III

# PERANCANGAN DAN IMPLEMENTASI

## 3.1. Langkah-langkah Program

Program dimulai dengan menerima dua input dari user: path file .obj yang ingin dikonversi dan kedalaman maksimum Octree. Program kemudian memvalidasi kedua input tersebut sebelum melanjutkan proses komputasi. Terdapat beberapa konsep yang perlu dipahami untuk memahami cara kerja program:

- Vertex: titik dalam ruang 3 dimensi yang didefinisikan oleh koordinat (x, y, z).
- Face: permukaan sisi yang dibentuk dari tiga vertex, dalam program ini selalu berupa segitiga.
- Mesh: struktur data yang menyimpan kumpulan vertices dan kumpulan segitiga dari sebuah model 3D.
- AABB: kotak pembatas terkecil yang sejajar dengan sumbu koordinat untuk membungkus seluruh mesh.
- SAT: algoritma pengujian perpotongan antara segitiga dan AABB menggunakan Separating Axis Theorem.

Cara kerja program secara keseluruhan dapat diringkas dalam langkah-langkah berikut:

1. User memasukkan path file .obj yang valid dan kedalaman maksimum Octree. Program memvalidasi keduanya: path harus menunjuk ke file yang ada dan berekstensikan .obj, sementara kedalaman harus berupa bilangan bulat positif.
2. Program mem-parsing file .obj baris per baris, mengekstrak baris vertex (v x y z) dan face (f i j k) menjadi struktur Mesh. Baris lain seperti vn, mtllib, usemtl, dan komentar diabaikan. Program juga menangani format face dengan slash seperti f 1/2/3.

3. Dihitung AABB terkecil yang melingkupi seluruh vertex mesh (computeAABB), kemudian diperluas menjadi kubus sempurna sebagai root Octree (expandToCube).
4. Node root diinisialisasi dengan AABB kubus ini dan seluruh indeks triangle dimasukkan ke dalamnya, lalu fungsi buildOctree dipanggil secara rekursif.
5. Pada setiap node yang belum mencapai kedalaman maksimum, node dibagi menjadi 8 oktan dengan membelah di titik tengah sumbu X, Y, Z. Setiap triangle dicek apakah berpotongan dengan tiap oktan menggunakan SAT. Triangle yang berpotongan diwarisi ke oktan tersebut.
6. Oktan yang tidak memiliki triangle langsung dipangkas (tidak dibagi lagi). Proses berlanjut rekursif hingga kedalaman maksimum tercapai.
7. Leaf node di kedalaman maksimum yang masih memiliki triangle dijadikan voxel aktif. Setiap voxel di-generate menjadi 8 vertex dan 12 segitiga (6 sisi × 2 segitiga per sisi), lalu seluruhnya ditulis ke file .obj output.
8. Program menampilkan laporan lengkap ke terminal: statistik node per depth, statistik pruning, jumlah voxel, vertex, face, waktu eksekusi, dan path output.
9. Setelah file output selesai ditulis, program membuka viewer 3D interaktif menggunakan SDL2 yang menampilkan hasil voxelisasi secara real-time dengan kontrol orbit (drag mouse) dan zoom (scroll).

## 3.2. Pseudocode Algoritma

### 3.2.1. Algoritma Pembangunan Octree (buildOctree)

```
PROCEDURE buildOctree(node: pointer to OctreeNode; mesh: Mesh;
maxDepth: integer)
```

```
DESKRIPSI:
  { Base case }
  if node.depth >= maxDepth OR node.triangleIndices kosong then
    node.isLeaf <- true
    return
  endif
```

```
  { Hitung titik tengah bounding box }
  mid.x <- (node.bounds.min.x + node.bounds.max.x) * 0.5
  mid.y <- (node.bounds.min.y + node.bounds.max.y) * 0.5
  mid.z <- (node.bounds.min.z + node.bounds.max.z) * 0.5

  { Tentukan 8 bounding box anak (oktan) }
  childBounds[0] <- AABB(node.bounds.min, mid)
    childBounds[1]  <-  AABB((mid.x,  min.y,  min.z),  (max.x,  mid.y,
mid.z))
    childBounds[2]  <-  AABB((min.x,  mid.y,  min.z),  (mid.x,  max.y,
mid.z))
    childBounds[3]  <-  AABB((mid.x,  mid.y,  min.z),  (max.x,  max.y,
mid.z))
    childBounds[4]  <-  AABB((min.x,  min.y,  mid.z),  (mid.x,  mid.y,
max.z))
    childBounds[5]  <-  AABB((mid.x,  min.y,  mid.z),  (max.x,  mid.y,
max.z))
    childBounds[6]  <-  AABB((min.x,  mid.y,  mid.z),  (mid.x,  max.y,
max.z))
  childBounds[7] <- AABB(mid, node.bounds.max)

  { Distribusikan triangle ke anak yang berpotongan }
  for i <- 0 to 7 do
    alokasikan node.children[i] sebagai OctreeNode baru
    node.children[i].bounds <- childBounds[i]
    node.children[i].depth  <- node.depth + 1
    for setiap idx in node.triangleIndices do
      v0, v1, v2 <- ambil vertex dari mesh.triangles[idx]
      if triangleIntersectsAABB(v0, v1, v2, childBounds[i]) then
        tambahkan idx ke node.children[i].triangleIndices
      endif
    endfor
    buildOctree(node.children[i], mesh, maxDepth)  { rekursi }
  endfor
  node.isLeaf <- false
```

### 3.2.2. Algoritma Pengujian Perpotongan SAT (triangleIntersectsAABB)

```
FUNCTION triangleIntersectsAABB(v0, v1, v2: Vec3; box: AABB) ->
boolean
```

```
DESKRIPSI:
  { Translasikan ke pusat box }
  center  <- (box.min + box.max) * 0.5
  extents <- (box.max - box.min) * 0.5
  v0 <- v0 - center;  v1 <- v1 - center;  v2 <- v2 - center
  e0 <- v1-v0;  e1 <- v2-v1;  e2 <- v0-v2

  { Uji 9 sumbu dari cross product edge x sumbu koordinat }
  for setiap kombinasi (ei, sumbu-koordinat) do
    axis <- cross(ei, sumbu-koordinat)
    p0, p2 <- proyeksikan v0, v2 ke axis
    r <- dot product extents dengan |axis| per komponen
    if max(-max(p0,p2), min(p0,p2)) > r then return false endif
  endfor

  { Uji 3 sumbu koordinat AABB }
    if  max(v0.x,v1.x,v2.x)  <  -extents.x  OR  min(v0.x,v1.x,v2.x)  >
extents.x
    then return false endif
  { ... idem untuk Y dan Z }

  { Uji 1 sumbu normal segitiga }
  normal <- cross(e0, e1)
  d  <- dot(normal, v0)
      rn   <-   extents.x*|normal.x|   +   extents.y*|normal.y|   +
extents.z*|normal.z|
  if d > rn OR d < -rn then return false endif

  return true
```

### 3.2.3. Algoritma Generate Geometri Voxel (generateVoxelMesh)

```
FUNCTION generateVoxelMesh(voxels: array of pointer to OctreeNode)
                          -> VoxelMesh
```

```
DESKRIPSI:
  { 12 face: 6 sisi x 2 segitiga per sisi, winding CCW dari luar }
  faceIndices <- { {0,2,1},{0,3,2}, {4,5,6},{4,6,7},
                   {0,1,5},{0,5,4}, {2,3,7},{2,7,6},
                   {0,4,7},{0,7,3}, {1,2,6},{1,6,5} }

  for n <- 0 to |voxels|-1 do
    mn <- voxels[n].bounds.min
    mx <- voxels[n].bounds.max
    { 8 vertex sudut kubus }
    tambahkan Vec3(mn.x,mn.y,mn.z), Vec3(mx.x,mn.y,mn.z),
              Vec3(mx.x,mx.y,mn.z), Vec3(mn.x,mx.y,mn.z),
              Vec3(mn.x,mn.y,mx.z), Vec3(mx.x,mn.y,mx.z),
              Vec3(mx.x,mx.y,mx.z), Vec3(mn.x,mx.y,mx.z)
    globalOffset <- n * 8 + 1  { 1-based untuk format OBJ }
    { 12 face dengan index global }
    for f <- 0 to 11 do
      tri.v[0..2] <- globalOffset + faceIndices[f][0..2]
      tambahkan tri ke result.faces
    endfor
  endfor
  return result
```

## 3.3. Implementasi Konkuren

### 3.3.1. Strategi Konkuren pada buildOctree

Pada proses pembangunan Octree, bagian yang paling berat secara komputasi adalah eksplorasi rekursif ke seluruh sub-pohon. Strategi yang diterapkan adalah spawning thread hanya di level pertama (depth 0 ke depth 1), di mana 8 anak dari node root diproses secara paralel menggunakan 8 std::thread yang berbeda.

Masing-masing thread kemudian melanjutkan rekursinya sendiri secara sequential ke level-level berikutnya tanpa spawn thread baru.

Alasan pemilihan strategi ini adalah untuk menghindari overhead pembuatan thread yang berlebihan. Apabila thread baru di-spawn di setiap level rekursi, jumlah thread yang aktif akan tumbuh secara eksponensial (8, 64, 512, ...) dan justru memperlambat program akibat overhead manajemen thread. Dengan hanya spawn di level pertama, diperoleh 8 thread paralel yang masing-masing mengerjakan 1/8 beban total secara efisien.

Karena setiap sub-pohon bekerja pada rangkaian node yang sepenuhnya terpisah (tidak ada node yang dibagi antara dua thread), tidak diperlukan mutex sama sekali pada bagian ini, sehingga tidak ada lock contention.

### 3.3.2. Strategi Konkuren pada generateVoxelMesh

Pada proses generate geometri voxel, setiap voxel adalah independen: vertex dan face voxel ke-n tidak bergantung pada voxel ke-(n-1) maupun ke-(n+1). Oleh karena itu, proses ini sangat ideal untuk diparalelkan.

Strategi yang diterapkan adalah membagi daftar voxels menjadi N chunk yang sama besar, di mana N ditentukan secara otomatis dari jumlah hardware thread yang tersedia (std::thread::hardware_concurrency()). Setiap thread memproses chunk-nya ke dalam buffer lokal sendiri (vector<Vec3> dan vector<Triangle> per thread), tanpa pernah menyentuh data thread lain.

Kunci keberhasilan strategi ini adalah penggunaan per-thread buffer. Index face global (globalOffset = n * 8 + 1) dihitung berdasarkan posisi absolut voxel di vector asli, bukan posisi relatif di dalam chunk, sehingga setiap thread dapat menghitung index yang benar secara independen tanpa perlu koordinasi antar thread. Mutex hanya digunakan satu kali di akhir setelah semua thread selesai, saat buffer-buffer lokal digabungkan ke result akhir.

Tabel berikut merangkum penerapan konkuren di kedua bagian:

| Bagian | Strategi | Sinkronisasi |
|---|---|---|
| buildOctree | 8 std::thread untuk 8 anak root, rekursi selanjutnya sequential | join() saja, tidak perlu mutex (data terpisah per sub-pohon) |
| generateVoxelMesh | N thread sesuai hardware_concurrency(), tiap thread proses satu chunk | Per-thread buffer lokal, merge ke result setelah join() |

### 3.3.3. Implementasi OBJ Viewer Interaktif

Sebagai bonus, program dilengkapi dengan viewer 3D interaktif menggunakan library SDL2. Viewer menampilkan hasil voxelisasi dalam bentuk wireframe dengan proyeksi perspektif sederhana. Viewer mendukung dua mode interaksi:

- Orbit: klik dan tahan tombol kiri mouse, lalu gerakkan untuk merotasikan model pada sumbu X dan Y.
- Zoom: scroll mouse ke atas untuk memperbesar tampilan, scroll ke bawah untuk memperkecil.

Viewer diimplementasikan sebagai struct Viewer dengan metode init() untuk inisialisasi window SDL2, run() untuk event loop utama, render() untuk menggambar wireframe per frame, dan close() untuk membersihkan resource. Proyeksi perspektif dilakukan secara manual dengan formula proyeksi standar: koordinat 2D dihitung dari koordinat 3D dengan membagi oleh (fov + z), menghasilkan efek perspektif sederhana namun cukup untuk visualisasi model 3D.

### 3.3.3 Source Code dan Pseudocode

3.3.3.1 Source Code

**types.hpp**

```
#pragma once

#include <vector>


struct Vec3

{

    float x, y, z;


    Vec3 operator+(Vec3 a) const

    {

        Vec3 res;

        res.x = x + a.x;

        res.y = y + a.y;

        res.z = z + a.z;

        return (res);

    }

    Vec3 operator-(Vec3 a) const

    {

        Vec3 res;

        res.x = x - a.x;

        res.y = y - a.y;

        res.z = z - a.z;

        return (res);
```

```
    }
    Vec3 operator+(float a) const
    {
        Vec3 res;
        res.x = x + a;
        res.y = y + a;
        res.z = z + a;
        return (res);
    }
    Vec3 operator-(float a) const
    {
        Vec3 res;
        res.x = x - a;
        res.y = y - a;
        res.z = z - a;
        return (res);
    }
    Vec3 operator*(float a) const
    {
        Vec3 res;
        res.x = x * a;
        res.y = y * a;
        res.z = z * a;
        return (res);
    }
};
```

```
struct Triangle
{
    int v[3];
};

struct Mesh
{
    std::vector<Vec3> vertices;
    std::vector<Triangle> triangles;
};

struct AABB
{
    Vec3 min;
    Vec3 max;
};

inline Vec3 cross(Vec3 a, Vec3 b)
{
     return {a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y -
a.y * b.x};
}

inline float dot(Vec3 a, Vec3 b)
{
```

```
    return a.x * b.x + a.y * b.y + a.z * b.z;
}

struct OctreeNode
{
    AABB bounds;
    int depth;
    OctreeNode *children[8];
    std::vector<int> triangleIndices;
    bool isLeaf;

    OctreeNode() : depth(0), isLeaf(false)
    {
        for (int i = 0; i < 8; i++)
            children[i] = nullptr;
    }
};

struct VoxelMesh
{
    std::vector<Vec3> vertices;
    std::vector<Triangle> faces;
};
```

**input.hpp**

```
#pragma once
```

```
#include <iostream>

#include <fstream>

#include <limits>

#include "types.hpp"



std::pair<std::string, int> inputObj()

{

    bool pass = false;

    std::string path;

    int maxDepth;

    while (!pass)

    {

        pass = true;


        std::cout << "Masukkan path obj file Anda: " << std::endl;

        std::cin >> path;

        std::ifstream f(path);

        if (f.good())

        {

            if (path.substr(path.size() - 4, 4).compare(".obj") != 0)

            {

                pass = false;

            }

        }

        else
```

```
        {
            pass = false;
        }

        if (!pass)
            std::cout << "Masukkan path yang valid\n\n";
    }
    pass = false;
    while (!pass)
    {
          std::cout << "Masukkan tingkat kedalaman pohon maksimum : " <<
std::endl;
        std::cin >> maxDepth;
        if (std::cin.fail())
        {
            std::cout << "Masukkan tingkat kedalaman yang valid\n";
            std::cin.clear();
             std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
            pass = false;
        }
        else if (maxDepth < 1)
        {
            std::cout << "Kedalaman minimal adalah 1!\n";
            pass = false;
        }
        else
```

```
        {
            pass = true;
        }
    }

    return std::make_pair(path, maxDepth);
}
```

**parser.hpp**

```
#pragma once
#include "types.hpp"
#include <algorithm>
#include <fstream>
#include <iostream>
#include <sstream>
#include <string>

Mesh validateObj(const std::string &path) {
  Mesh mesh;
  std::ifstream file(path);
  std::string line;
  int lineNum = 0;
  while (std::getline(file, line)) {
    lineNum++;
    std::istringstream iss(line);
    std::string token;
```

```
    iss >> token;

     if (token != "#" && !token.empty() && token != "vn" && token !=
"fn") {

      if (token == "v") {

        float x, y, z;

        if (!(iss >> x >> y >> z)) {

          std::cerr << "Error baris " << lineNum

                          << "Input obj tidak valid program gagal. Terima
kasih! "

                       "Baca lagi spesifikasi ya"

                    << std::endl;

          exit(1);

        }


        mesh.vertices.push_back({x, y, z});

      } else if (token == "f") {

        std::string s1, s2, s3;

        Triangle temp;


        if (!(iss >> s1 >> s2 >> s3)) {

          std::cerr << "Error. gagal membaca.";

          continue;

        }


        size_t pos1 = s1.find('/');

        std::string justNum1;

        if (pos1 != std::string::npos) {
```

```
        justNum1 = s1.substr(0, pos1);
      } else {
        justNum1 = s1;
      }

      int index1 = std::stoi(justNum1);

      size_t pos2 = s2.find('/');
      std::string justNum2;
      if (pos2 != std::string::npos) {
        justNum2 = s2.substr(0, pos2);
      } else {
        justNum2 = s2;
      }

      int index2 = std::stoi(justNum2);

      size_t pos3 = s3.find('/');
      std::string justNum3;
      if (pos3 != std::string::npos) {
        justNum3 = s3.substr(0, pos3);
      } else {
        justNum3 = s3;
      }

      int index3 = std::stoi(justNum3);
```

```
        temp.v[0] = index1 - 1;
        temp.v[1] = index2 - 1;
        temp.v[2] = index3 - 1;
        mesh.triangles.push_back(temp);
      }
    }
  }
  return mesh;
}

AABB computeAABB(const Mesh &mesh) {
    if (mesh.vertices.empty()) {
        std::cerr << "Error: mesh tidak memiliki vertex." << std::endl;
        exit(1);
    }
  AABB res;
  Vec3 min = mesh.vertices[0];
  Vec3 max = mesh.vertices[0];
  for (int i = 1; i < mesh.vertices.size(); i++) {
    min.x = std::min(min.x, mesh.vertices[i].x);
    min.y = std::min(min.y, mesh.vertices[i].y);
    min.z = std::min(min.z, mesh.vertices[i].z);
    max.x = std::max(max.x, mesh.vertices[i].x);
    max.y = std::max(max.y, mesh.vertices[i].y);
    max.z = std::max(max.z, mesh.vertices[i].z);
```

```
  }
  res.min = min;
  res.max = max;
  return res;
}

AABB expandToCube(AABB aabb) {
  float dx = aabb.max.x - aabb.min.x;
  float dy = aabb.max.y - aabb.min.y;
  float dz = aabb.max.z - aabb.min.z;
  float side = std::max({dx, dy, dz});
  Vec3 center = (aabb.min + aabb.max) * 0.5f;
  aabb.min = center - side * 0.5f;
  aabb.max = center + side * 0.5f;
  return aabb;
}
```

**octree.hpp**

```
#pragma once
#include "types.hpp"
#include <cmath>
#include <map>
#include <algorithm>
#include <thread>

inline bool triangleIntersectsAABB(Vec3 v0, Vec3 v1, Vec3 v2, const
```

```
AABB& box) {
        Vec3 center = {(box.min.x + box.max.x) * 0.5f, (box.min.y +
box.max.y) * 0.5f, (box.min.z + box.max.z) * 0.5f};
        Vec3 extents = {(box.max.x - box.min.x) * 0.5f, (box.max.y -
box.min.y) * 0.5f, (box.max.z - box.min.z) * 0.5f};

    v0 = v0 - center;
    v1 = v1 - center;
    v2 = v2 - center;

    Vec3 e0 = v1 - v0;
    Vec3 e1 = v2 - v1;
    Vec3 e2 = v0 - v2;

    float p0, p1, p2, r;

    float fex = std::fabs(e0.x);
    float fey = std::fabs(e0.y);
    float fez = std::fabs(e0.z);

    p0 = e0.z * v0.y - e0.y * v0.z;
    p2 = e0.z * v2.y - e0.y * v2.z;
    r = fez * extents.y + fey * extents.z;
    if (std::max(-std::max(p0, p2), std::min(p0, p2)) > r) return false;

    p0 = -e0.z * v0.x + e0.x * v0.z;
    p2 = -e0.z * v2.x + e0.x * v2.z;
```

```
    r = fez * extents.x + fex * extents.z;
    if (std::max(-std::max(p0, p2), std::min(p0, p2)) > r) return false;

    p0 = e0.y * v0.x - e0.x * v0.y;
    p2 = e0.y * v2.x - e0.x * v2.y;
    r = fey * extents.x + fex * extents.y;
    if (std::max(-std::max(p0, p2), std::min(p0, p2)) > r) return false;

    fex = std::fabs(e1.x);
    fey = std::fabs(e1.y);
    fez = std::fabs(e1.z);

    p0 = e1.z * v0.y - e1.y * v0.z;
    p2 = e1.z * v2.y - e1.y * v2.z;
    r = fez * extents.y + fey * extents.z;
    if (std::max(-std::max(p0, p2), std::min(p0, p2)) > r) return false;

    p0 = -e1.z * v0.x + e1.x * v0.z;
    p2 = -e1.z * v2.x + e1.x * v2.z;
    r = fez * extents.x + fex * extents.z;
    if (std::max(-std::max(p0, p2), std::min(p0, p2)) > r) return false;

    p0 = e1.y * v0.x - e1.x * v0.y;
    p2 = e1.y * v2.x - e1.x * v2.y;
    r = fey * extents.x + fex * extents.y;
    if (std::max(-std::max(p0, p2), std::min(p0, p2)) > r) return false;
```

```
    fex = std::fabs(e2.x);
    fey = std::fabs(e2.y);
    fez = std::fabs(e2.z);

    p0 = e2.z * v0.y - e2.y * v0.z;
    p1 = e2.z * v1.y - e2.y * v1.z;
    r = fez * extents.y + fey * extents.z;
    if (std::max(-std::max(p0, p1), std::min(p0, p1)) > r) return false;

    p0 = -e2.z * v0.x + e2.x * v0.z;
    p1 = -e2.z * v1.x + e2.x * v1.z;
    r = fez * extents.x + fex * extents.z;
    if (std::max(-std::max(p0, p1), std::min(p0, p1)) > r) return false;

    p0 = e2.y * v0.x - e2.x * v0.y;
    p1 = e2.y * v1.x - e2.x * v1.y;
    r = fey * extents.x + fex * extents.y;
    if (std::max(-std::max(p0, p1), std::min(p0, p1)) > r) return false;

      if (std::max({v0.x, v1.x, v2.x}) < -extents.x || std::min({v0.x,
v1.x, v2.x}) > extents.x) return false;
      if (std::max({v0.y, v1.y, v2.y}) < -extents.y || std::min({v0.y,
v1.y, v2.y}) > extents.y) return false;
      if (std::max({v0.z, v1.z, v2.z}) < -extents.z || std::min({v0.z,
v1.z, v2.z}) > extents.z) return false;
```

```
    Vec3 normal = cross(e0, e1);

    float d = dot(normal, v0);

        float rn = extents.x * std::fabs(normal.x) + extents.y *
std::fabs(normal.y) + extents.z * std::fabs(normal.z);

    if (d > rn || d < -rn) return false;


    return true;

}


inline void buildOctreeSequential(OctreeNode* node, const Mesh& mesh,
int maxDepth) {

    if (node->depth >= maxDepth || node->triangleIndices.empty()) {

        node->isLeaf = true;

        return;

    }


    Vec3 mid = {

        (node->bounds.min.x + node->bounds.max.x) * 0.5f,

        (node->bounds.min.y + node->bounds.max.y) * 0.5f,

        (node->bounds.min.z + node->bounds.max.z) * 0.5f

    };


    AABB childBounds[8];

    childBounds[0] = {node->bounds.min, mid};

     childBounds[1] = {{mid.x, node->bounds.min.y, node->bounds.min.z},
{node->bounds.max.x, mid.y, mid.z}};

     childBounds[2] = {{node->bounds.min.x, mid.y, node->bounds.min.z},
```

```
{mid.x, node->bounds.max.y, mid.z}};

            childBounds[3]   =   {{mid.x,   mid.y,   node->bounds.min.z},
{node->bounds.max.x, node->bounds.max.y, mid.z}};

     childBounds[4]  =  {{node->bounds.min.x, node->bounds.min.y, mid.z},
{mid.x, mid.y, node->bounds.max.z}};

            childBounds[5]   =   {{mid.x,   node->bounds.min.y,   mid.z},
{node->bounds.max.x, mid.y, node->bounds.max.z}};

       childBounds[6]  =  {{node->bounds.min.x,  mid.y,  mid.z},  {mid.x,
node->bounds.max.y, node->bounds.max.z}};

    childBounds[7] = {mid, node->bounds.max};


    for (int i = 0; i < 8; i++) {

        node->children[i] = new OctreeNode();

        node->children[i]->bounds = childBounds[i];

        node->children[i]->depth = node->depth + 1;


        for (int idx : node->triangleIndices) {

            const Triangle& tri = mesh.triangles[idx];

            Vec3 v0 = mesh.vertices[tri.v[0]];

            Vec3 v1 = mesh.vertices[tri.v[1]];

            Vec3 v2 = mesh.vertices[tri.v[2]];

            if (triangleIntersectsAABB(v0, v1, v2, childBounds[i])) {

                node->children[i]->triangleIndices.push_back(idx);

            }

        }


        buildOctreeSequential(node->children[i], mesh, maxDepth);
```

```
    }

    node->isLeaf = false;
}

inline void buildOctree(OctreeNode* node, const Mesh& mesh, int maxDepth) {
    if (node->depth >= maxDepth || node->triangleIndices.empty()) {
        node->isLeaf = true;
        return;
    }

    Vec3 mid = {
        (node->bounds.min.x + node->bounds.max.x) * 0.5f,
        (node->bounds.min.y + node->bounds.max.y) * 0.5f,
        (node->bounds.min.z + node->bounds.max.z) * 0.5f
    };

    AABB childBounds[8];
    childBounds[0] = {node->bounds.min, mid};
    childBounds[1] = {{mid.x, node->bounds.min.y, node->bounds.min.z}, {node->bounds.max.x, mid.y, mid.z}};
    childBounds[2] = {{node->bounds.min.x, mid.y, node->bounds.min.z}, {mid.x, node->bounds.max.y, mid.z}};
    childBounds[3] = {{mid.x, mid.y, node->bounds.min.z}, {node->bounds.max.x, node->bounds.max.y, mid.z}};
    childBounds[4] = {{node->bounds.min.x, node->bounds.min.y, mid.z}, {mid.x, mid.y, node->bounds.max.z}};
```

```
            childBounds[5]   =   {{mid.x,   node->bounds.min.y,   mid.z},
{node->bounds.max.x, mid.y, node->bounds.max.z}};

        childBounds[6]  =  {{node->bounds.min.x,  mid.y,  mid.z},  {mid.x,
node->bounds.max.y, node->bounds.max.z}};

    childBounds[7] = {mid, node->bounds.max};


    for (int i = 0; i < 8; i++) {

        node->children[i] = new OctreeNode();

        node->children[i]->bounds = childBounds[i];

        node->children[i]->depth = node->depth + 1;


        for (int idx : node->triangleIndices) {

            const Triangle& tri = mesh.triangles[idx];

            Vec3 v0 = mesh.vertices[tri.v[0]];

            Vec3 v1 = mesh.vertices[tri.v[1]];

            Vec3 v2 = mesh.vertices[tri.v[2]];

            if (triangleIntersectsAABB(v0, v1, v2, childBounds[i])) {

                node->children[i]->triangleIndices.push_back(idx);

            }

        }

    }


    std::thread threads[8];

    for (int i = 0; i < 8; i++) {

                        threads[i]   =   std::thread(buildOctreeSequential,
node->children[i], std::cref(mesh), maxDepth);

    }
```

```
    for (int i = 0; i < 8; i++) {
        threads[i].join();
    }

    node->isLeaf = false;
}

inline void collectStats(OctreeNode* node, std::map<int, int>&
nodeCount, std::map<int, int>& prunedCount) {
    if (node == nullptr) return;
    nodeCount[node->depth]++;
    if (node->isLeaf && node->triangleIndices.empty()) {
        prunedCount[node->depth]++;
    }
    for (int i = 0; i < 8; i++) {
        collectStats(node->children[i], nodeCount, prunedCount);
    }
}

inline std::vector<OctreeNode*> collectLeafVoxels(OctreeNode* node) {
    std::vector<OctreeNode*> result;
    if (node == nullptr) return result;
    if (node->isLeaf) {
        if (!node->triangleIndices.empty()) {
            result.push_back(node);
        }
```

```
        return result;

    }

    for (int i = 0; i < 8; i++) {

                                std::vector<OctreeNode*>    childResult    =
collectLeafVoxels(node->children[i]);

                        result.insert(result.end(),   childResult.begin(),
childResult.end());

    }

    return result;

}
```

**output.hpp**

```
#pragma once

#include <iostream>

#include <fstream>

#include <limits>

#include "types.hpp"

#include <string>

#include <filesystem>

#include <cstdio>

#include <thread>

namespace fs = std::filesystem;


VoxelMesh generateVoxelMesh(const std::vector<OctreeNode*>& voxels) {

    int faceIndices[12][3] = {

        {0,2,1}, {0,3,2},

        {4,5,6}, {4,6,7},
```

```
        {0,1,5}, {0,5,4},
        {2,3,7}, {2,7,6},
        {0,4,7}, {0,7,3},
        {1,2,6}, {1,6,5}
    };

    unsigned int numThreads = std::thread::hardware_concurrency();
    if (numThreads == 0) numThreads = 4;
    int totalVoxels = (int)voxels.size();
    if ((int)numThreads > totalVoxels) numThreads = totalVoxels;

    std::vector<std::vector<Vec3>> threadVertices(numThreads);
    std::vector<std::vector<Triangle>> threadFaces(numThreads);

    std::vector<int> chunkStart(numThreads);
    std::vector<int> chunkEnd(numThreads);
    int baseSize = totalVoxels / numThreads;
    int remainder = totalVoxels % numThreads;
    int offset = 0;
    for (unsigned int t = 0; t < numThreads; t++) {
        chunkStart[t] = offset;
        int thisSize = baseSize + (t < (unsigned int)remainder ? 1 : 0);
        chunkEnd[t] = offset + thisSize;
        offset += thisSize;
    }
```

```
    std::vector<std::thread> threads(numThreads);

    for (unsigned int t = 0; t < numThreads; t++) {

        threads[t] = std::thread([&, t]() {

            for (int n = chunkStart[t]; n < chunkEnd[t]; n++) {

                Vec3 mn = voxels[n]->bounds.min;

                Vec3 mx = voxels[n]->bounds.max;


                threadVertices[t].push_back({mn.x, mn.y, mn.z});

                threadVertices[t].push_back({mx.x, mn.y, mn.z});

                threadVertices[t].push_back({mx.x, mx.y, mn.z});

                threadVertices[t].push_back({mn.x, mx.y, mn.z});

                threadVertices[t].push_back({mn.x, mn.y, mx.z});

                threadVertices[t].push_back({mx.x, mn.y, mx.z});

                threadVertices[t].push_back({mx.x, mx.y, mx.z});

                threadVertices[t].push_back({mn.x, mx.y, mx.z});


                int globalOffset = n * 8 + 1;

                for (int f = 0; f < 12; f++) {

                    Triangle tri;

                    tri.v[0] = globalOffset + faceIndices[f][0];

                    tri.v[1] = globalOffset + faceIndices[f][1];

                    tri.v[2] = globalOffset + faceIndices[f][2];

                    threadFaces[t].push_back(tri);

                }

            }

        });
```

```
    }


    for (unsigned int t = 0; t < numThreads; t++) {
        threads[t].join();
    }


    VoxelMesh result;
    for (unsigned int t = 0; t < numThreads; t++) {
                            result.vertices.insert(result.vertices.end(),
threadVertices[t].begin(), threadVertices[t].end());
         result.faces.insert(result.faces.end(), threadFaces[t].begin(),
threadFaces[t].end());
    }
    return result;
}


void objWriter(VoxelMesh res, const std::string &input_path, const int
&depth)
{
    fs::path p(input_path);
    std::string directory = p.parent_path().string();
    std::string stem = p.stem().string();
       std::string output_filename = stem + "_voxelized_max-depth_" +
std::to_string(depth)+".obj";
      std::string output_path = directory.empty() ? output_filename :
directory + "/" + output_filename;


    std::ofstream output_file(output_path);
```

```
    if (!output_file.is_open())
    {
        std::cerr << "Error: Could not open the file." << std::endl;
        return;
    }

    for(int i = 0; i < (int)res.vertices.size(); i++){
                output_file << "v " << res.vertices[i].x << ' ' <<
res.vertices[i].y << ' ' << res.vertices[i].z << '\n';
    }

    for (int i = 0; i < (int)res.faces.size(); i++)
    {
                output_file << "f " << res.faces[i].v[0] << ' ' <<
res.faces[i].v[1] << ' ' << res.faces[i].v[2] << '\n';
    }

    output_file.close();
    std::cout << "Output disimpan di      : " << output_path << '\n';
}
```

**viewer.hpp**

```
#pragma once
#include <SDL.h>
#include "types.hpp"
#include <vector>
#include <iostream>
```

```
#include <cmath>


    struct Viewer{

    SDL_Window* window = nullptr;

    SDL_Renderer* renderer = nullptr;


    int height= 600;

    int width = 800;


    float fov = 300.0f;

    float anglex = 0.0f;

    float angley = 0.0f;

    float zoom = 1.0f;

    Vec3 centerOffset = {0.0f, 0.0f, 0.0f};



    bool init(){

        if(SDL_Init(SDL_INIT_VIDEO) != 0){

            std::cerr << "SDL gagal: " << SDL_GetError() << "\n";

            return false;


        }

         window = SDL_CreateWindow("3D Renderer", SDL_WINDOWPOS_CENTERED,
SDL_WINDOWPOS_CENTERED, 800, 600, 0);

        if(window == nullptr) {

            std::cerr << "SDL gagal: " << SDL_GetError() << "\n";
```

```
            return false;
        }
                        renderer   =   SDL_CreateRenderer(window,   -1,
SDL_RENDERER_ACCELERATED);
        if(renderer == nullptr){
            std::cerr << "SDL gagal: " << SDL_GetError() << "\n";
            return false;
        }
        return true;
    }


    void autofit(const VoxelMesh& mesh) {
        if (mesh.vertices.empty()) return;
        Vec3 mn = mesh.vertices[0], mx = mesh.vertices[0];
        for (const Vec3& v : mesh.vertices) {
            if (v.x < mn.x) mn.x = v.x; if (v.x > mx.x) mx.x = v.x;
            if (v.y < mn.y) mn.y = v.y; if (v.y > mx.y) mx.y = v.y;
            if (v.z < mn.z) mn.z = v.z; if (v.z > mx.z) mx.z = v.z;
        }
        centerOffset.x = (mn.x + mx.x) * 0.5f;
        centerOffset.y = (mn.y + mx.y) * 0.5f;
        centerOffset.z = (mn.z + mx.z) * 0.5f;
        float maxSpan = std::max({mx.x - mn.x, mx.y - mn.y, mx.z - mn.z});
        if (maxSpan > 0.0f)
            zoom = (std::min(width, height) * 0.7f) / maxSpan;
    }
```

```
    Vec3 rotatedVertex(Vec3& v){
        Vec3 tmp;
        tmp.x = v.
x * cos(angley) + v.z * sin(angley);
        tmp.z = -v.x * sin(angley) + v.z * cos(angley);
        tmp.y = v.y;

        Vec3 rotated;
        rotated.x = tmp.x;
        rotated.y = tmp.y * cos(anglex) - tmp.z * sin(anglex);
        rotated.z = tmp.y * sin(anglex) + tmp.z * cos(anglex);
        return rotated;
    }

    SDL_Point project(Vec3& v){
        SDL_Point projected;
        float projected_x = v.x * zoom * fov/(fov + v.z);
        float projected_y = v.y * zoom * fov/(fov + v.z);
        projected.x = (int) (projected_x + width/2);
        projected.y = (int) (projected_y + height/2);
        return projected;
    }

    void render(const VoxelMesh &mesh)
```

```
    {
        SDL_SetRenderDrawColor(renderer, 0, 0, 0, 255);
        SDL_RenderClear(renderer);

        SDL_SetRenderDrawColor(renderer, 255, 255, 255, 255);
        for (const Triangle &tri : mesh.faces)
        {
            Vec3 vertex1 = mesh.vertices[tri.v[0]];
            Vec3 vertex2 = mesh.vertices[tri.v[1]];
            Vec3 vertex3 = mesh.vertices[tri.v[2]];

            Vec3 rotatedVertex1 = rotatedVertex(vertex1);
            Vec3 rotatedVertex2 = rotatedVertex(vertex2);
            Vec3 rotatedVertex3 = rotatedVertex(vertex3);

            SDL_Point project1 = project(rotatedVertex1);
            SDL_Point project2 = project(rotatedVertex2);
            SDL_Point project3 = project(rotatedVertex3);

                    SDL_RenderDrawLine(renderer, project1.x, project1.y,
project2.x, project2.y);
                    SDL_RenderDrawLine(renderer, project2.x, project2.y,
project3.x, project3.y);
                    SDL_RenderDrawLine(renderer, project3.x, project3.y,
project1.x, project1.y);
        }
```

```
        SDL_RenderPresent(renderer);
    }

    void run(const VoxelMesh &mesh)
    {
        bool running = true;
        bool mouseDown = false;
        int lastMouseX = 0, lastMouseY = 0;

        autofit(mesh);
        while(running){
            SDL_Event event;

            while(SDL_PollEvent(&event)){
                switch (event.type)
                {
                case SDL_QUIT:
                    running = false;
                    break;
                case SDL_MOUSEBUTTONDOWN:
                    if(event.button.button == SDL_BUTTON_LEFT){
                        mouseDown = true;
                        lastMouseX = event.button.x;
                        lastMouseY = event.button.y;
                    }
                    break;
```

```
                case SDL_MOUSEBUTTONUP:
                    if(event.button.button ==SDL_BUTTON_LEFT){
                        mouseDown = false;
                    }
                    break;
                case SDL_MOUSEMOTION:
                    if(mouseDown){
                        int dx = event.motion.x - lastMouseX;
                        int dy = event.motion.y - lastMouseY;

                        angley += dx * 0.01;
                        anglex += dy * 0.01;

                        lastMouseX = event.motion.x;
                        lastMouseY = event.motion.y;
                    }
                    break;
                case SDL_MOUSEWHEEL:
                    if(event.wheel.y > 0) zoom += 0.5f;
                      else if(event.wheel.y < 0)   zoom = std::max(0.1f,
zoom - 0.5f);
                    break;



                }
            }
```

```
            render(mesh);
            SDL_Delay(16);
        }
    }

    void close()
    {
        SDL_DestroyRenderer(renderer);
        SDL_DestroyWindow(window);
        SDL_Quit();
    }
};
```

**main.cpp**

```
#include <iostream>
#include <map>
#include <chrono>
#include "input.hpp"
#include "parser.hpp"
#include "octree.hpp"
#include "output.hpp"
#include <utility>
#include "viewer.hpp"

int main() {
    std::pair<std::string, int> userData = inputObj();
```

```
    std::string pathFile = userData.first;
    int depth = userData.second;

    auto startTime = std::chrono::high_resolution_clock::now();

    Mesh mesh = validateObj(pathFile);

    AABB initialSquare = computeAABB(mesh);
    AABB finalSquare = expandToCube(initialSquare);

     std::cout << "Jumlah Vertex (input)    : " << mesh.vertices.size() <<
std::endl;
     std::cout << "Jumlah Triangle (input)  : " << mesh.triangles.size()
<< std::endl;
     std::cout << "Bounding Box             : " << finalSquare.min.x << ",
"
              << finalSquare.min.y << ", " << finalSquare.min.z << ", "
              << finalSquare.max.x << ", " << finalSquare.max.y << ", "
              << finalSquare.max.z << std::endl;

    OctreeNode* root = new OctreeNode();
    root->bounds = finalSquare;
    root->depth = 0;
    for (int i = 0; i < (int)mesh.triangles.size(); i++) {
        root->triangleIndices.push_back(i);
    }
```

```
    buildOctree(root, mesh, depth);

    std::map<int, int> nodeCount, prunedCount;
    collectStats(root, nodeCount, prunedCount);

    std::cout << "Statistik node octree:" << std::endl;
    for (auto& p : nodeCount) {
        std::cout << p.first << " : " << p.second << std::endl;
    }

    std::cout << "Statistik node di-prune:" << std::endl;
    for (auto& p : prunedCount) {
        std::cout << p.first << " : " << p.second << std::endl;
    }

    std::cout << "Kedalaman octree        : " << depth << std::endl;

    std::vector<OctreeNode*> voxels = collectLeafVoxels(root);
     std::cout << "Jumlah voxel                : " << voxels.size() <<
std::endl;

    VoxelMesh voxelMesh = generateVoxelMesh(voxels);



           std::cout << "Jumlah vertex output          : " <<
voxelMesh.vertices.size() << std::endl;
```

```
    std::cout << "Jumlah face output       : " << voxelMesh.faces.size()
<< std::endl;


    objWriter(voxelMesh, pathFile, depth);


    auto endTime = std::chrono::high_resolution_clock::now();
                                        auto          duration          =
std::chrono::duration_cast<std::chrono::milliseconds>(endTime          -
startTime);
    std::cout << "Waktu eksekusi           : " << duration.count() << "
ms" << std::endl;


    Viewer view;
    if(view.init()){
        view.run(voxelMesh);
        view.close();
    }


    return 0;
}
```

3.3.3.2 Pseudocode

| **types.hpp** |
| --- |
| `{ Definisi tipe-tipe dasar yang digunakan seluruh program }` |

```
TYPE Vec3 : < x, y, z : real >

TYPE Triangle : < v : array[0..2] of integer >

TYPE Mesh :
    < vertices  : array of Vec3
      triangles : array of Triangle >

TYPE AABB :
    < min : Vec3
      max : Vec3 >

TYPE OctreeNode :
    < bounds          : AABB
      depth           : integer
      children        : array[0..7] of pointer to OctreeNode
      triangleIndices : array of integer
      isLeaf          : boolean >

TYPE VoxelMesh :
    < vertices : array of Vec3
      faces    : array of Triangle >

{ Konstruktor default OctreeNode }
PROCEDURE initOctreeNode(output node : OctreeNode)
DESKRIPSI
    node.depth  <- 0
    node.isLeaf <- false
    for i <- 0 to 7 do
        node.children[i] <- nil
    endfor
```

```
{ Operasi vektor }
FUNCTION addVec3(a, b : Vec3) -> Vec3
DEKLARASI
    res : Vec3
DESKRIPSI
    res.x <- a.x + b.x
    res.y <- a.y + b.y
    res.z <- a.z + b.z
    -> res

FUNCTION subVec3(a, b : Vec3) -> Vec3
DEKLARASI
    res : Vec3
DESKRIPSI
    res.x <- a.x - b.x
    res.y <- a.y - b.y
    res.z <- a.z - b.z
    -> res

FUNCTION scaleVec3(a : Vec3; s : real) -> Vec3
DEKLARASI
    res : Vec3
DESKRIPSI
    res.x <- a.x * s
    res.y <- a.y * s
    res.z <- a.z * s
    -> res

FUNCTION cross(a, b : Vec3) -> Vec3
DESKRIPSI
```

```
    -> Vec3(a.y*b.z - a.z*b.y,
            a.z*b.x - a.x*b.z,
            a.x*b.y - a.y*b.x)

FUNCTION dot(a, b : Vec3) -> real
DESKRIPSI
    -> a.x*b.x + a.y*b.y + a.z*b.z
```

**input.hpp**

```
{ Membaca path file .obj dan kedalaman maksimum octree dari user }

FUNCTION inputObj() -> < path : string, maxDepth : integer >
DEKLARASI
    pass     : boolean
    path     : string
    maxDepth : integer
    f        : file
DESKRIPSI
    { Validasi path file .obj }
    pass <- false
    while not pass do
        pass <- true
        output("Masukkan path obj file Anda: ")
        input(path)

        buka file f dengan path
        if file f tidak dapat dibuka then
            pass <- false
        else if 4 karakter terakhir path ≠ ".obj" then
```

```
            pass <- false
        endif

        if not pass then
            output("Masukkan path yang valid")
        endif
    endwhile

    { Validasi kedalaman maksimum }
    pass <- false
    while not pass do
        output("Masukkan tingkat kedalaman pohon maksimum: ")
        input(maxDepth)

        if input gagal (bukan integer) then
            output("Masukkan tingkat kedalaman yang valid")
            bersihkan buffer input
            pass <- false
        else if maxDepth < 1 then
            output("Kedalaman minimal adalah 1!")
            pass <- false
        else
            pass <- true
        endif
    endwhile

    -> (path, maxDepth)
```

**parser.hpp**

```
{ Membaca dan memvalidasi file .obj, menghitung AABB dan kubus
pembatas }


FUNCTION validateObj(path : string) -> Mesh
DEKLARASI
    mesh    : Mesh
    file    : file teks
    line    : string
    lineNum : integer
    token   : string
    x, y, z : real
    s1, s2, s3 : string
    temp    : Triangle
    pos, index1, index2, index3 : integer
DESKRIPSI
    buka file dengan path
    lineNum <- 0

    while masih ada baris di file do
        baca baris -> line
        lineNum <- lineNum + 1
        baca token pertama dari line -> token

        if token = "v" then
            if tidak bisa baca 3 bilangan real (x, y, z) then
                  output("Error baris ", lineNum, ": Input obj tidak
valid")
                keluar program
            endif
            tambahkan Vec3(x, y, z) ke mesh.vertices
```

```
        else if token = "f" then
            baca s1, s2, s3 dari sisa baris
            if gagal membaca then
                output("Error: gagal membaca face")
                lanjutkan ke iterasi berikutnya
            endif

            { Ambil indeks vertex sebelum karakter '/' jika ada }
            index1 <- ambilIndeks(s1)
            index2 <- ambilIndeks(s2)
            index3 <- ambilIndeks(s3)

             temp.v[0] <- index1 - 1   { konversi 1-based ke 0-based
}
            temp.v[1] <- index2 - 1
            temp.v[2] <- index3 - 1
            tambahkan temp ke mesh.triangles
        endif
        { abaikan baris yang diawali '#', 'vn', 'fn', atau kosong }
    endwhile

    -> mesh

{ Fungsi pembantu: ambil indeks vertex dari string "i/j/k" atau "i"
}
FUNCTION ambilIndeks(s : string) -> integer
DEKLARASI
    pos : integer
DESKRIPSI
    pos <- posisi karakter '/' pertama dalam s
    if pos ditemukan then
```

```
        -> stoi(substring s dari 0 sampai pos)
    else
        -> stoi(s)
    endif

FUNCTION computeAABB(mesh : Mesh) -> AABB
DEKLARASI
    res      : AABB
    minV, maxV : Vec3
DESKRIPSI
    if mesh.vertices kosong then
        output("Error: mesh tidak memiliki vertex")
        keluar program
    endif

    minV <- mesh.vertices[0]
    maxV <- mesh.vertices[0]

    for i <- 1 to |mesh.vertices| - 1 do
        minV.x <- min(minV.x, mesh.vertices[i].x)
        minV.y <- min(minV.y, mesh.vertices[i].y)
        minV.z <- min(minV.z, mesh.vertices[i].z)
        maxV.x <- max(maxV.x, mesh.vertices[i].x)
        maxV.y <- max(maxV.y, mesh.vertices[i].y)
        maxV.z <- max(maxV.z, mesh.vertices[i].z)
    endfor

    res.min <- minV
    res.max <- maxV
    -> res
```

```
FUNCTION expandToCube(aabb : AABB) -> AABB
DEKLARASI
    dx, dy, dz : real
    side       : real
    center     : Vec3
DESKRIPSI
    dx <- aabb.max.x - aabb.min.x
    dy <- aabb.max.y - aabb.min.y
    dz <- aabb.max.z - aabb.min.z
    side <- max(dx, dy, dz)

    center <- scaleVec3(addVec3(aabb.min, aabb.max), 0.5)

    aabb.min <- subVec3(center, Vec3(side*0.5, side*0.5, side*0.5))
    aabb.max <- addVec3(center, Vec3(side*0.5, side*0.5, side*0.5))
    -> aabb
```

**octree.hpp**

```
{ Pengujian perpotongan segitiga dengan Axis-Aligned Bounding Box
  menggunakan Teorema Sumbu Pemisah (SAT - Separating Axis Theorem)
}

FUNCTION triangleIntersectsAABB(v0, v1, v2 : Vec3; box : AABB) ->
boolean
DEKLARASI
    center, extents : Vec3
    e0, e1, e2      : Vec3
    p0, p1, p2, r   : real
    fex, fey, fez   : real
    normal          : Vec3
```

```
    d, rn           : real
DESKRIPSI
    center  <- scaleVec3(addVec3(box.min, box.max), 0.5)
    extents <- scaleVec3(subVec3(box.max, box.min), 0.5)

    { Translasikan segitiga ke pusat box }
    v0 <- subVec3(v0, center)
    v1 <- subVec3(v1, center)
    v2 <- subVec3(v2, center)

    { Hitung vektor sisi segitiga }
    e0 <- subVec3(v1, v0)
    e1 <- subVec3(v2, v1)
    e2 <- subVec3(v0, v2)

     { === Uji 9 sumbu dari cross product sisi segitiga dengan sumbu
koordinat === }

    { Sumbu-sumbu dari e0 }
    fex <- |e0.x|; fey <- |e0.y|; fez <- |e0.z|

    p0 <- e0.z*v0.y - e0.y*v0.z
    p2 <- e0.z*v2.y - e0.y*v2.z
    r  <- fez*extents.y + fey*extents.z
    if max(-max(p0,p2), min(p0,p2)) > r then -> false endif

    p0 <- -e0.z*v0.x + e0.x*v0.z
    p2 <- -e0.z*v2.x + e0.x*v2.z
    r  <- fez*extents.x + fex*extents.z
    if max(-max(p0,p2), min(p0,p2)) > r then -> false endif
```

```
p0 <- e0.y*v0.x - e0.x*v0.y
p2 <- e0.y*v2.x - e0.x*v2.y
r  <- fey*extents.x + fex*extents.y
if max(-max(p0,p2), min(p0,p2)) > r then -> false endif

{ Sumbu-sumbu dari e1 }
fex <- |e1.x|; fey <- |e1.y|; fez <- |e1.z|

p0 <- e1.z*v0.y - e1.y*v0.z
p2 <- e1.z*v2.y - e1.y*v2.z
r  <- fez*extents.y + fey*extents.z
if max(-max(p0,p2), min(p0,p2)) > r then -> false endif

p0 <- -e1.z*v0.x + e1.x*v0.z
p2 <- -e1.z*v2.x + e1.x*v2.z
r  <- fez*extents.x + fex*extents.z
if max(-max(p0,p2), min(p0,p2)) > r then -> false endif

p0 <- e1.y*v0.x - e1.x*v0.y
p2 <- e1.y*v2.x - e1.x*v2.y
r  <- fey*extents.x + fex*extents.y
if max(-max(p0,p2), min(p0,p2)) > r then -> false endif

{ Sumbu-sumbu dari e2 }
fex <- |e2.x|; fey <- |e2.y|; fez <- |e2.z|

p0 <- e2.z*v0.y - e2.y*v0.z
p1 <- e2.z*v1.y - e2.y*v1.z
r  <- fez*extents.y + fey*extents.z
if max(-max(p0,p1), min(p0,p1)) > r then -> false endif
```

```
    p0 <- -e2.z*v0.x + e2.x*v0.z
    p1 <- -e2.z*v1.x + e2.x*v1.z
    r  <- fez*extents.x + fex*extents.z
    if max(-max(p0,p1), min(p0,p1)) > r then -> false endif

    p0 <- e2.y*v0.x - e2.x*v0.y
    p1 <- e2.y*v1.x - e2.x*v1.y
    r  <- fey*extents.x + fex*extents.y
    if max(-max(p0,p1), min(p0,p1)) > r then -> false endif

    { === Uji 3 sumbu koordinat (AABB face normals) === }
     if max(v0.x, v1.x, v2.x) < -extents.x OR min(v0.x, v1.x, v2.x)
> extents.x then -> false endif
     if max(v0.y, v1.y, v2.y) < -extents.y OR min(v0.y, v1.y, v2.y)
> extents.y then -> false endif
     if max(v0.z, v1.z, v2.z) < -extents.z OR min(v0.z, v1.z, v2.z)
> extents.z then -> false endif

    { === Uji 1 sumbu normal segitiga === }
    normal <- cross(e0, e1)
    d  <- dot(normal, v0)
          rn  <-  extents.x*|normal.x|  +  extents.y*|normal.y|  +
extents.z*|normal.z|
    if d > rn OR d < -rn then -> false endif

    -> true

{ ----------------------------------------------------------------
  buildOctreeSequential: rekursi penuh pada satu thread
    ----------------------------------------------------------------
}
PROCEDURE buildOctreeSequential(node : pointer to OctreeNode;
                                mesh : Mesh; maxDepth : integer)
```

```
DEKLARASI
    mid         : Vec3
    childBounds : array[0..7] of AABB
    i, idx      : integer
    tri         : Triangle
    v0, v1, v2  : Vec3
DESKRIPSI
      { Basis: node adalah daun jika kedalaman maks tercapai atau
tidak ada segitiga }
    if node.depth >= maxDepth OR node.triangleIndices kosong then
        node.isLeaf <- true
        return
    endif

    { Hitung titik tengah bounding box }
    mid.x <- (node.bounds.min.x + node.bounds.max.x) * 0.5
    mid.y <- (node.bounds.min.y + node.bounds.max.y) * 0.5
    mid.z <- (node.bounds.min.z + node.bounds.max.z) * 0.5

    { Tentukan 8 bounding box anak (oktan) }
    childBounds[0] <- AABB(node.bounds.min, mid)
          childBounds[1]   <-   AABB(Vec3(mid.x,   node.bounds.min.y,
node.bounds.min.z),
                           Vec3(node.bounds.max.x, mid.y, mid.z))
          childBounds[2]   <-   AABB(Vec3(node.bounds.min.x,   mid.y,
node.bounds.min.z),
                           Vec3(mid.x, node.bounds.max.y, mid.z))
    childBounds[3] <- AABB(Vec3(mid.x, mid.y, node.bounds.min.z),
                                            Vec3(node.bounds.max.x,
node.bounds.max.y, mid.z))
                 childBounds[4]    <-    AABB(Vec3(node.bounds.min.x,
node.bounds.min.y, mid.z),
                           Vec3(mid.x, mid.y, node.bounds.max.z))
```

```
    childBounds[5] <- AABB(Vec3(mid.x, node.bounds.min.y, mid.z),
                                        Vec3(node.bounds.max.x,  mid.y,
node.bounds.max.z))
    childBounds[6] <- AABB(Vec3(node.bounds.min.x, mid.y, mid.z),
                                        Vec3(mid.x,  node.bounds.max.y,
node.bounds.max.z))
    childBounds[7] <- AABB(mid, node.bounds.max)

    { Buat node anak dan distribusikan segitiga }
    for i <- 0 to 7 do
        alokasikan node.children[i] sebagai OctreeNode baru
        node.children[i].bounds <- childBounds[i]
        node.children[i].depth  <- node.depth + 1

        for setiap idx in node.triangleIndices do
            tri <- mesh.triangles[idx]
            v0  <- mesh.vertices[tri.v[0]]
            v1  <- mesh.vertices[tri.v[1]]
            v2  <- mesh.vertices[tri.v[2]]
              if triangleIntersectsAABB(v0, v1, v2, childBounds[i])
then
                tambahkan idx ke node.children[i].triangleIndices
            endif
        endfor

        { Rekursi ke anak }
        buildOctreeSequential(node.children[i], mesh, maxDepth)
    endfor

    node.isLeaf <- false

{ ----------------------------------------------------------------
```

```
  buildOctree: level pertama diparalelkan (8 thread),
               selanjutnya sekuensial
    ----------------------------------------------------------------
}
PROCEDURE buildOctree(node : pointer to OctreeNode;
                      mesh : Mesh; maxDepth : integer)
DEKLARASI
    mid         : Vec3
    childBounds : array[0..7] of AABB
    i, idx      : integer
    tri         : Triangle
    v0, v1, v2  : Vec3
    threads     : array[0..7] of Thread
DESKRIPSI
    if node.depth >= maxDepth OR node.triangleIndices kosong then
        node.isLeaf <- true
        return
    endif

    mid.x <- (node.bounds.min.x + node.bounds.max.x) * 0.5
    mid.y <- (node.bounds.min.y + node.bounds.max.y) * 0.5
    mid.z <- (node.bounds.min.z + node.bounds.max.z) * 0.5

     { Tentukan childBounds[0..7] sama seperti buildOctreeSequential
}
    { ... (definisi 8 oktan identik dengan di atas) ... }

     { Distribusikan segitiga ke anak, lalu jalankan rekursi secara
paralel }
    for i <- 0 to 7 do
        alokasikan node.children[i]
        node.children[i].bounds <- childBounds[i]
```

```
        node.children[i].depth  <- node.depth + 1

        for setiap idx in node.triangleIndices do
            tri <- mesh.triangles[idx]
            v0  <- mesh.vertices[tri.v[0]]
            v1  <- mesh.vertices[tri.v[1]]
            v2  <- mesh.vertices[tri.v[2]]
              if triangleIntersectsAABB(v0, v1, v2, childBounds[i])
then
                tambahkan idx ke node.children[i].triangleIndices
            endif
        endfor
    endfor

    { Jalankan buildOctreeSequential untuk tiap anak secara paralel
}
    for i <- 0 to 7 do
        threads[i] <- Thread baru yang menjalankan:
            buildOctreeSequential(node.children[i], mesh, maxDepth)
    endfor

    { Tunggu semua thread selesai }
    for i <- 0 to 7 do
        join(threads[i])
    endfor

    node.isLeaf <- false

{ Kumpulkan statistik jumlah node per kedalaman }
PROCEDURE collectStats(node       : pointer to OctreeNode;
                       nodeCount  : map<integer,integer>;
```

```
                        prunedCount: map<integer,integer>)
DESKRIPSI
    if node = nil then return endif

    nodeCount[node.depth] <- nodeCount[node.depth] + 1

    if node.isLeaf AND node.triangleIndices kosong then
        prunedCount[node.depth] <- prunedCount[node.depth] + 1
    endif

    for i <- 0 to 7 do
        collectStats(node.children[i], nodeCount, prunedCount)
    endfor

{ Kumpulkan semua leaf node yang mengandung segitiga (= voxel
aktif) }
FUNCTION collectLeafVoxels(node : pointer to OctreeNode)
    -> array of pointer to OctreeNode
DEKLARASI
    result      : array of pointer to OctreeNode
    childResult : array of pointer to OctreeNode
DESKRIPSI
    if node = nil then -> result endif

    if node.isLeaf then
        if node.triangleIndices tidak kosong then
            tambahkan node ke result
        endif
        -> result
    endif
```

```
    for i <- 0 to 7 do
        childResult <- collectLeafVoxels(node.children[i])
        gabungkan childResult ke result
    endfor

    -> result
```

**output.hpp**

```
{ Hasilkan mesh kubus voxel dari kumpulan leaf node octree }


FUNCTION generateVoxelMesh(voxels : array of pointer to OctreeNode)
-> VoxelMesh
DEKLARASI
    { 12 face (sepasang segitiga per 6 sisi kubus) }
    faceIndices : array[0..11][0..2] of integer
        <- { {0,2,1},{0,3,2},  { sisi belakang  }
             {4,5,6},{4,6,7},  { sisi depan     }
             {0,1,5},{0,5,4},  { sisi bawah     }
             {2,3,7},{2,7,6},  { sisi atas      }
             {0,4,7},{0,7,3},  { sisi kiri      }
             {1,2,6},{1,6,5} } { sisi kanan     }

    numThreads          : integer
    totalVoxels         : integer
    threadVertices      : array of (array of Vec3)
    threadFaces         : array of (array of Triangle)
    chunkStart, chunkEnd: array of integer
    baseSize, remainder : integer
    threads             : array of Thread
```

```
    result              : VoxelMesh
    mn, mx              : Vec3
    globalOffset        : integer
    tri                 : Triangle
DESKRIPSI
    numThreads  <- jumlah core hardware (minimal 4)
    totalVoxels <- |voxels|
      if numThreads > totalVoxels then numThreads <- totalVoxels
endif

    { Bagi rentang voxel ke tiap thread secara merata }
    baseSize  <- totalVoxels div numThreads
    remainder <- totalVoxels mod numThreads
    offset    <- 0
    for t <- 0 to numThreads-1 do
        chunkStart[t] <- offset
        thisSize <- baseSize + (if t < remainder then 1 else 0)
        chunkEnd[t] <- offset + thisSize
        offset <- offset + thisSize
    endfor

    { Tiap thread memproses chunk-nya masing-masing }
    for t <- 0 to numThreads-1 do
        threads[t] <- Thread baru yang menjalankan:
            for n <- chunkStart[t] to chunkEnd[t]-1 do
                mn <- voxels[n].bounds.min
                mx <- voxels[n].bounds.max

                { 8 vertex sudut kubus }
                            tambahkan Vec3(mn.x, mn.y, mn.z) ke
threadVertices[t]
                            tambahkan Vec3(mx.x, mn.y, mn.z) ke
```

```
threadVertices[t]
                              tambahkan  Vec3(mx.x,  mx.y,  mn.z)  ke
threadVertices[t]
                              tambahkan  Vec3(mn.x,  mx.y,  mn.z)  ke
threadVertices[t]
                              tambahkan  Vec3(mn.x,  mn.y,  mx.z)  ke
threadVertices[t]
                              tambahkan  Vec3(mx.x,  mn.y,  mx.z)  ke
threadVertices[t]
                              tambahkan  Vec3(mx.x,  mx.y,  mx.z)  ke
threadVertices[t]
                              tambahkan  Vec3(mn.x,  mx.y,  mx.z)  ke
threadVertices[t]


                  globalOffset <- n * 8 + 1   { indeks 1-based untuk
format OBJ }


                { 12 segitiga face }
                for f <- 0 to 11 do
                    tri.v[0] <- globalOffset + faceIndices[f][0]
                    tri.v[1] <- globalOffset + faceIndices[f][1]
                    tri.v[2] <- globalOffset + faceIndices[f][2]
                    tambahkan tri ke threadFaces[t]
                endfor
            endfor
    endfor

    for t <- 0 to numThreads-1 do
        join(threads[t])
    endfor

    { Gabungkan hasil semua thread }
    for t <- 0 to numThreads-1 do
```

```
        gabungkan threadVertices[t] ke result.vertices
        gabungkan threadFaces[t]    ke result.faces
    endfor

    -> result

{ Tulis VoxelMesh ke file .obj }
PROCEDURE objWriter(res        : VoxelMesh;
                    input_path : string;
                    depth      : integer)
DEKLARASI
    directory       : string
    stem            : string
    output_filename : string
    output_path     : string
    output_file     : file teks
DESKRIPSI
    directory <- direktori dari input_path
    stem      <- nama file tanpa ekstensi

     output_filename <- stem + "_voxelized_max-depth_" + str(depth)
+ ".obj"
    output_path     <- if directory kosong then output_filename
                       else directory + "/" + output_filename

    buka output_file di output_path untuk ditulis
    if gagal membuka then
        output("Error: Could not open the file.")
        return
    endif
```

```
    { Tulis vertex }
    for i <- 0 to |res.vertices|-1 do
                tulis ke output_file: "v ", res.vertices[i].x,
res.vertices[i].y, res.vertices[i].z
    endfor

    { Tulis face }
    for i <- 0 to |res.faces|-1 do
                tulis ke output_file: "f ", res.faces[i].v[0],
res.faces[i].v[1], res.faces[i].v[2]
    endfor

    tutup output_file
    output("Output disimpan di      : ", output_path)
```

**viewer.hpp**

```
{ Viewer 3D interaktif menggunakan SDL2 dengan proyeksi perspektif
sederhana }

TYPE Viewer :
    < window   : pointer to SDL_Window
      renderer : pointer to SDL_Renderer
      height   : integer  { = 600 }
      width    : integer  { = 800 }
      fov      : real     { = 300.0 }
      anglex   : real     { = 0.0 }
      angley   : real     { = 0.0 }
      zoom     : real     { = 1.0 } >

FUNCTION Viewer.init() -> boolean
DESKRIPSI
    if SDL_Init(VIDEO) gagal then
```

```
        output("SDL gagal: ", pesan error)
        -> false
    endif

    buat window SDL berukuran 800x600 berjudul "3D Renderer"
    if gagal then -> false endif

    buat renderer akselerasi hardware untuk window
    if gagal then -> false endif

    -> true

{ Rotasi vertex dengan sudut angley (sumbu Y) lalu anglex (sumbu X)
}
FUNCTION Viewer.rotatedVertex(v : Vec3) -> Vec3
DEKLARASI
    tmp, rotated : Vec3
DESKRIPSI
    { Rotasi terhadap sumbu Y }
    tmp.x <- v.x * cos(angley) + v.z * sin(angley)
    tmp.z <- -v.x * sin(angley) + v.z * cos(angley)
    tmp.y <- v.y

    { Rotasi terhadap sumbu X }
    rotated.x <- tmp.x
    rotated.y <- tmp.y * cos(anglex) - tmp.z * sin(anglex)
    rotated.z <- tmp.y * sin(anglex) + tmp.z * cos(anglex)
    -> rotated

{ Proyeksi perspektif dari koordinat 3D ke piksel 2D }
FUNCTION Viewer.project(v : Vec3) -> Point2D
```

```
DEKLARASI
    projected : Point2D
DESKRIPSI
    projected.x <- (int)(v.x * zoom * fov / (fov + v.z) + width/2)
    projected.y <- (int)(v.y * zoom * fov / (fov + v.z) + height/2)
    -> projected

{ Render satu frame: gambar wireframe semua segitiga }
PROCEDURE Viewer.render(mesh : VoxelMesh)
DEKLARASI
    v1, v2, v3  : Vec3
    r1, r2, r3  : Vec3
    p1, p2, p3  : Point2D
DESKRIPSI
    bersihkan layar dengan warna hitam

    set warna gambar ke putih
    for setiap tri in mesh.faces do
        v1 <- mesh.vertices[tri.v[0]]
        v2 <- mesh.vertices[tri.v[1]]
        v3 <- mesh.vertices[tri.v[2]]

        r1 <- rotatedVertex(v1)
        r2 <- rotatedVertex(v2)
        r3 <- rotatedVertex(v3)

        p1 <- project(r1)
        p2 <- project(r2)
        p3 <- project(r3)

        gambar garis p1 -> p2
```

```
        gambar garis p2 -> p3
        gambar garis p3 -> p1
    endfor

    tampilkan frame ke layar

{ Loop utama: tangani event & render terus-menerus }
PROCEDURE Viewer.run(mesh : VoxelMesh)
DEKLARASI
    running    : boolean <- true
    mouseDown  : boolean <- false
    lastMouseX : integer <- 0
    lastMouseY : integer <- 0
    dx, dy     : integer
DESKRIPSI
    while running do
        { Proses semua event dalam antrian }
        while ada event do
            baca event

            if event = QUIT then
                running <- false

            else if event = MOUSE_BUTTON_DOWN (kiri) then
                mouseDown  <- true
                lastMouseX <- event.x
                lastMouseY <- event.y

            else if event = MOUSE_BUTTON_UP (kiri) then
                mouseDown <- false
```

```
            else if event = MOUSE_MOTION AND mouseDown then
                dx <- event.x - lastMouseX
                dy <- event.y - lastMouseY
                angley <- angley + dx * 0.01
                anglex <- anglex + dy * 0.01
                lastMouseX <- event.x
                lastMouseY <- event.y

            else if event = MOUSE_WHEEL then
                if scroll ke atas then
                    zoom <- zoom + 0.5
                else if scroll ke bawah then
                    zoom <- max(0.1, zoom - 0.5)
                endif
            endif
        endwhile

        render(mesh)
    endwhile

PROCEDURE Viewer.close()
DESKRIPSI
    hapus renderer
    hapus window
    SDL_Quit()
```

**main.cpp**

```
ALGORITMA VoxelisasiOBJ


DEKLARASI
```

```
    userData      : < path : string, maxDepth : integer >
    pathFile      : string
    depth         : integer
    startTime     : waktu
    endTime       : waktu
    duration      : integer { dalam milidetik }

    mesh          : Mesh
    initialAABB   : AABB
    finalAABB     : AABB

    root          : pointer to OctreeNode

    nodeCount     : map<integer, integer>
    prunedCount   : map<integer, integer>
    p             : pasangan <integer, integer>

    voxels        : array of pointer to OctreeNode
    voxelMesh     : VoxelMesh

    view          : Viewer

DESKRIPSI
    { === Fase 1: Input === }
    userData <- inputObj()
    pathFile <- userData.path
    depth    <- userData.maxDepth

    startTime <- waktu sekarang

    { === Fase 2: Parsing === }
```

```
mesh <- validateObj(pathFile)

{ === Fase 3: Hitung Bounding Box Kubik === }
initialAABB <- computeAABB(mesh)
finalAABB   <- expandToCube(initialAABB)

output("Jumlah Vertex (input)   : ", |mesh.vertices|)
output("Jumlah Triangle (input) : ", |mesh.triangles|)
output("Bounding Box            : ",
       finalAABB.min.x, finalAABB.min.y, finalAABB.min.z,
       finalAABB.max.x, finalAABB.max.y, finalAABB.max.z)

{ === Fase 4: Bangun Octree (Divide and Conquer) === }
alokasikan root sebagai OctreeNode baru
root.bounds <- finalAABB
root.depth  <- 0
for i <- 0 to |mesh.triangles|-1 do
    tambahkan i ke root.triangleIndices
endfor

buildOctree(root, mesh, depth)

{ === Fase 5: Kumpulkan Statistik === }
collectStats(root, nodeCount, prunedCount)

output("Statistik node octree:")
for setiap p in nodeCount do
    output(p.key, " : ", p.value)
endfor

output("Statistik node di-prune:")
```

```
for setiap p in prunedCount do
    output(p.key, " : ", p.value)
endfor

output("Kedalaman octree        : ", depth)

{ === Fase 6: Kumpulkan Voxel & Buat Mesh Output === }
voxels    <- collectLeafVoxels(root)
output("Jumlah voxel            : ", |voxels|)

voxelMesh <- generateVoxelMesh(voxels)

output("Jumlah vertex output    : ", |voxelMesh.vertices|)
output("Jumlah face output      : ", |voxelMesh.faces|)

{ === Fase 7: Tulis File OBJ === }
objWriter(voxelMesh, pathFile, depth)

endTime  <- waktu sekarang
duration <- selisih(endTime, startTime) dalam milidetik
output("Waktu eksekusi          : ", duration, " ms")

{ === Fase 8: Visualisasi Interaktif === }
if view.init() then
    view.run(voxelMesh)
    view.close()
endif
```

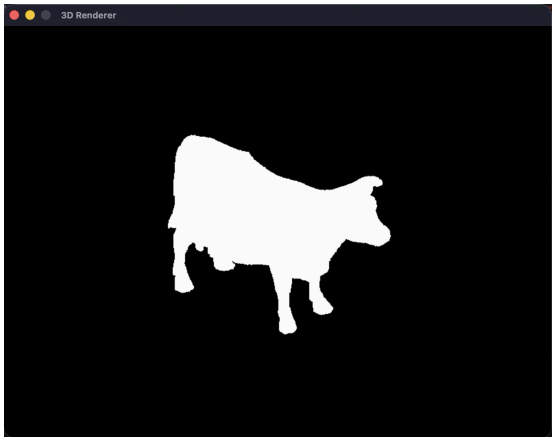
# BAB III

# PENGUJIAN

## 2.1. Pengujian Algoritma Divide and Conquer

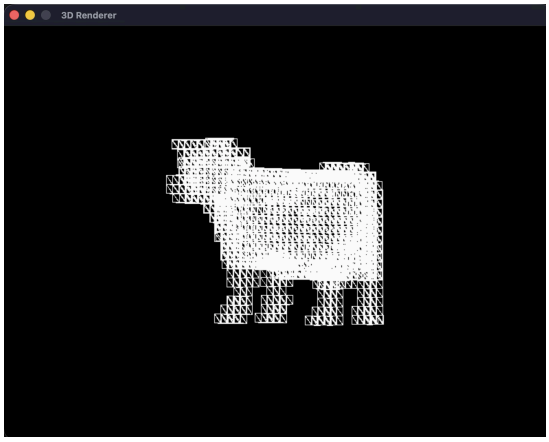
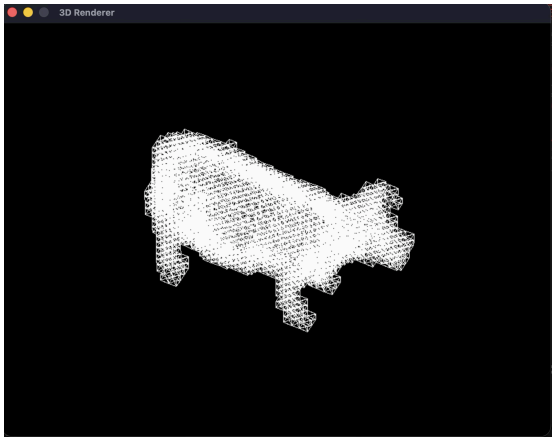
### 2.1.1 Kasus Uji 1

| | |
|---|---|
| **Nama File** | cow.obj |
| **Max Depth** | 8 |
| **Output CLI** | Jumlah Vertex (input) : 4583<br>Jumlah Triangle (input) : 5804<br>Bounding Box : -4.44584, -5.66062, -5.22196, 5.99809, 4.7833, 5.22196<br>Statistik node octree:<br>0 : 1<br>1 : 8<br>2 : 64<br>3 : 160<br>4 : 704<br>5 : 2656<br>6 : 10896<br>7 : 45016<br>8 : 182912<br>Statistik node di-prune:<br>2 : 44<br>3 : 72<br>4 : 372<br>5 : 1294<br>6 : 5269<br>7 : 22152<br>8 : 90811<br>Kedalaman octree : 8<br>Jumlah voxel : 92101<br>Jumlah vertex output : 736808<br>Jumlah face output : 1105212<br>Output disimpan di : cow_voxelized_max-depth_8.obj<br>Waktu eksekusi : 454 ms |

| | |
|---|---|
| **Tampilan Object Viewer** | <br>  |

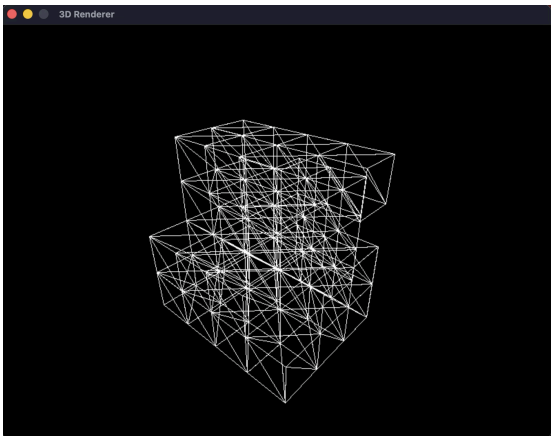
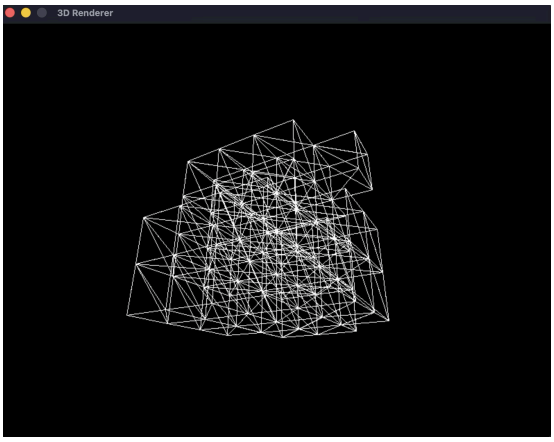
### 2.1.2 Kasus Uji 2

| | |
|---|---|
| **Nama File** | cow.obj |
| **Max Depth** | 5 |
| **Output CLI** | Jumlah Vertex (input) : 4583<br>Jumlah Triangle (input) : 5804<br>Bounding Box : -4.44584, -5.66062, -5.22196, 5.99809, 4.7833, 5.22196<br>Statistik node octree:<br>0 : 1<br>1 : 8<br>2 : 64<br>3 : 160<br>4 : 704<br>5 : 2656 |

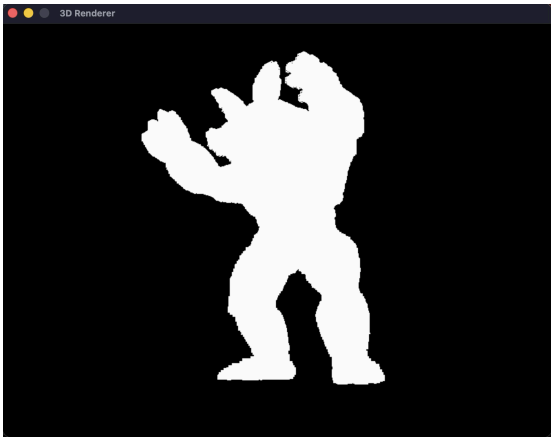
| | |
|---|---|
| | Statistik node di-prune:<br>2 : 44<br>3 : 72<br>4 : 372<br>5 : 1294<br>Kedalaman octree : 5<br>Jumlah voxel : 1362<br>Jumlah vertex output : 10896<br>Jumlah face output : 16344<br>Output disimpan di : cow_voxelized_max-depth_5.obj<br>Waktu eksekusi : 28 ms |
| **Tampilan Object Viewer** |  <br>  |

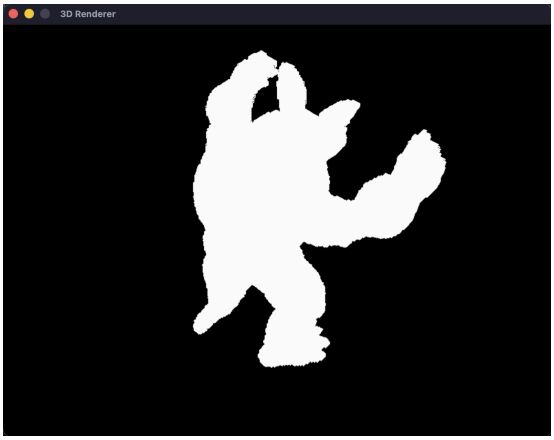
### 2.1.3 Kasus Uji 3

| | |
|---|---|
| **Nama File** | armadillo.obj |
| **Max Depth** | 2 |
| **Output CLI** | Jumlah Vertex (input) : 49990 |

| | |
|---|---|
| | Jumlah Triangle (input) : 99976<br>Bounding Box : -75.6434, -54.22, -75.6625, 75.6636, 97.087, 75.6445<br>Statistik node octree:<br>0 : 1<br>1 : 8<br>2 : 64<br>Statistik node di-prune:<br>2 : 22<br>Kedalaman octree : 2<br>Jumlah voxel : 42<br>Jumlah vertex output : 336<br>Jumlah face output : 504<br>Output disimpan di : armadillo_voxelized_max-depth_2.obj<br>Waktu eksekusi : 93 ms |
| **Tampilan Object Viewer** |  <br>  |

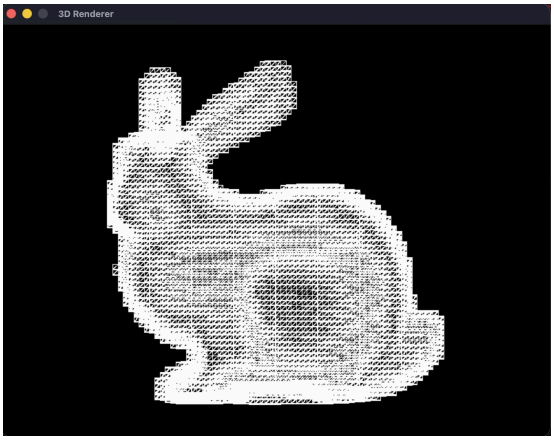
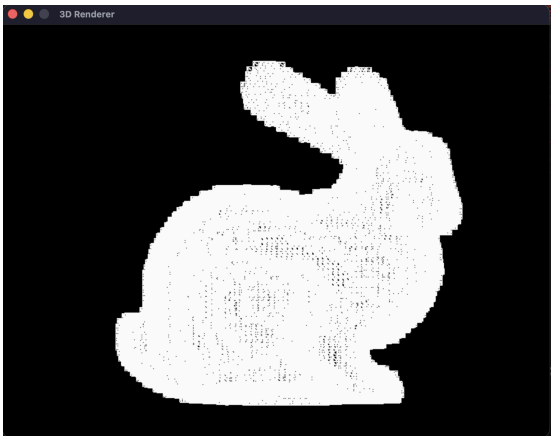
### 2.1.4 Kasus Uji 4

| | |
|---|---|
| **Nama File** | armadillol.obj |
| **Max Depth** | 7 |
| **Output CLI** | Jumlah Vertex (input) : 49990<br>Jumlah Triangle (input) : 99976<br>Bounding Box : -75.6434, -54.22, -75.6625, 75.6636, 97.087, 75.6445<br>Statistik node octree:<br>0 : 1<br>1 : 8<br>2 : 64<br>3 : 336<br>4 : 1224<br>5 : 4672<br>6 : 19496<br>7 : 79328<br>Statistik node di-prune:<br>2 : 22<br>3 : 183<br>4 : 640<br>5 : 2235<br>6 : 9580<br>7 : 38985<br>Kedalaman octree : 7<br>Jumlah voxel : 40343<br>Jumlah vertex output : 322744<br>Jumlah face output : 484116<br>Output disimpan di : armadillo_voxelized_max-depth_7.obj<br>Waktu eksekusi : 292 ms |
| **Tampilan Object Viewer** |  |

| | |
|---|---|
| |  |

### 2.1.5 Kasus Uji 5

| | |
|---|---|
| **Nama File** | bunny.obj |
| **Max Depth** | 6 |
| **Output CLI** | Jumlah Vertex (input) : 2503<br>Jumlah Triangle (input) : 4968<br>Bounding Box : -0.0943804, 0.0325733, -0.0790619, 0.0607788, 0.187733, 0.0760973<br>Statistik node octree:<br>0 : 1<br>1 : 8<br>2 : 64<br>3 : 336<br>4 : 1560<br>5 : 6648<br>6 : 27376<br>Statistik node di-prune:<br>2 : 22<br>3 : 141<br>4 : 729<br>5 : 3226<br>6 : 13463<br>Kedalaman octree : 6<br>Jumlah voxel : 13913<br>Jumlah vertex output : 111304<br>Jumlah face output : 166956<br>Output disimpan di : bunny_voxelized_max-depth_6.obj<br>Waktu eksekusi : 99 ms |

| | |
|---|---|
| **Tampilan Object Viewer** |  <br>  |

## 2.2. Analisis Hasil Pengujian

# BAB IV

## 3.1. Kesimpulan

Pada tugas besar ini, telah dilakukan implementasi dan analisis terhadap program voxelisasi objek 3D menggunakan struktur data Octree dengan pendekatan algoritma Divide and Conquer. Berdasarkan hasil implementasi yang telah dikembangkan, dapat disimpulkan bahwa penggunaan algoritma Divide and Conquer cukup efektif untuk memecah persoalan voxelisasi yang kompleks menjadi sub-masalah yang lebih kecil dan dapat diselesaikan secara independen.

Pendekatan ini memungkinkan program untuk membagi ruang tiga dimensi secara rekursif menjadi 8 oktan pada setiap level, sehingga proses penentuan voxel yang berpotongan dengan permukaan objek dapat dilakukan secara terstruktur dan efisien.

Dari pengamatan terhadap implementasi yang dibuat, terlihat bahwa algoritma Divide and Conquer tidak hanya diterapkan dalam bentuk pembagian ruang sederhana, tetapi juga dipadukan dengan mekanisme yang lebih kompleks, seperti pengujian perpotongan segitiga menggunakan Separating Axis Theorem (SAT), pruning pada oktan kosong, serta paralelisasi pada level pertama pembangunan Octree. Hal ini menunjukkan bahwa algoritma Divide and Conquer tetap relevan dan dapat dikembangkan lebih lanjut meskipun persoalan yang dihadapi bersifat geometris dan multidimensi, selama strategi pembagian dan penggabungan solusinya dirancang dengan tepat. Dalam konteks ini, efektivitas program tidak hanya ditentukan oleh struktur rekursinya, melainkan juga oleh kualitas uji perpotongan dan pengelolaan node yang dilakukan pada setiap tahap pembagian.

Meskipun program yang dibuat telah memberikan hasil yang cukup baik, masih terdapat beberapa kekurangan yang ditemukan dalam penerapannya, yaitu sebagai berikut.

1. Pertumbuhan node yang eksponensial Jumlah node Octree tumbuh secara eksponensial seiring bertambahnya kedalaman maksimum. Meskipun pruning membantu mengurangi node yang tidak perlu, pada kedalaman besar program masih membutuhkan memori dan waktu komputasi yang cukup besar, sehingga penggunaan pada model dengan geometri yang sangat kompleks perlu diperhatikan.
2. Voxelisasi hanya pada permukaan objek Program saat ini hanya mendeteksi voxel yang berpotongan dengan permukaan mesh (segitiga), bukan mengisi volume interior objek. Akibatnya, objek hasil voxelisasi bersifat berongga di bagian dalam, yang mungkin tidak sesuai untuk kasus penggunaan tertentu yang membutuhkan representasi volumetrik penuh.

3. Keterbatasan format input Program hanya mendukung file .obj dengan face berbentuk segitiga. File .obj dengan face berupa poligon lebih dari tiga sisi, atau format model 3D lainnya, belum dapat diproses tanpa modifikasi tambahan pada bagian parser.

Secara keseluruhan, penerapan algoritma Divide and Conquer melalui struktur Octree dalam program voxelisasi ini memberikan hasil yang cukup baik, terutama untuk model 3D dengan kompleksitas geometri sedang dan kedalaman octree yang terkontrol. Algoritma ini terbukti mampu menghasilkan representasi voxel yang akurat dan adaptif, khususnya ketika dipadukan dengan pengujian SAT, mekanisme pruning, dan paralelisasi thread. Namun, untuk memperoleh performa yang lebih stabil pada model yang lebih kompleks atau kedalaman yang lebih besar, implementasi masih perlu dikembangkan lebih lanjut dengan mekanisme yang mampu mengelola pertumbuhan node secara lebih efisien serta mendukung voxelisasi volumetrik penuh.

## Lampiran

**Pernyataan tidak melakukan kecurangan yang ditandatangani**

| |
|---|
| Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri. |

Hakam Avicena Mustain

Rava Khoman Tuah Saragih

**Tautan repository GitHub dan video**

Tautan GitHub: https://github.com/hakamavicena/Tucil2_13524075_13524107

**Tabel Penyelesaian spesifikasi wajib dan bonus**

| No | Poin | Ya | Tidak |
|---|---|---|---|
| 1 | Program berhasil di kompilasi tanpa kesalahan | ✓ | |
| 2 | Program berhasil dijalankan | ✓ | |
| 3 | Hasil konversi .obj terdiri dari kubus-kubus kecil | ✓ | |
| 4 | Program informasi-informasi atau report | ✓ | |
| 5 | Program dapat menerima masukan .obj, dan mengembalikan output | ✓ | |
| 6 | [Bonus] Program menggunakan *concurrency* | ✓ | |

| | | | |
|---|---|---|---|
| 7 | [Bonus] Program dapat menampilkan visualisasi .obj | ✓ | |

## Daftar Pustaka


Munir, R. (2026). *Algoritma Divide and Conquer (Bagian 1)*. Institut Teknologi Bandung. Diperoleh dari https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/07-Algoritma-Divide-and-Conquer-(2026)-Bagian1.pdf

Munir, R. (2026). *Algoritma Divide and Conquer (Bagian 2)*. Institut Teknologi Bandung. Diperoleh dari

https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/08-Algoritma-Divide-and-Conquer-(2026)-Bagian2.pdf

Munir, R. (2026). *Algoritma Divide and Conquer (Bagian 3)*. Institut Teknologi Bandung. Diperoleh dari https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/09-Algoritma-Divide-and-Conquer-(2026)-Bagian3.pdf

Munir, R. (2026). *Algoritma Divide and Conquer (Bagian 4)*. Institut Teknologi Bandung. Diperoleh dari https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/10-Algoritma-Divide-and-Conquer-(2026)-Bagian4.pdf

GraverDev. (t.th.). *How to make a 3D Renderer [Explained Simply]* [Video]. YouTube. *How to make a 3D Renderer [Explained Simply]*

Paper. (t.th.). *Massive Octree LOD Voxel Terrain [Tutorial] - Explanation* [Video]. YouTube. *Massive Octree LOD Voxel Terrain [Tutorial] - Explanation*

Racicot, G. (t.th.). *Octree algorithm visualization & explanation* [Video]. YouTube. *Octree algorithm visualization & explanation*